

Chapter 2: Objects and Handles

Windows is an object-based operating system, exposing various types of objects (usually referred to as *kernel Objects*), that provide the bulk of the functionality in Windows. Example object types are processes, threads and files. In this chapter we'll discuss the general theory related to kernel objects without too much details of any specific object type. The following chapters will go into details of many of these types.

In this chapter:

- Kernel Objects
 - Handles
 - Creating Objects
 - Object Names
 - Sharing Kernel Objects
 - Private Object Namespaces
-

Kernel Objects

The Windows kernel exposes various types of objects for use by user mode processes, the kernel itself and kernel mode drivers. Instances of these types are data structures in system (kernel) space, created and managed by the Object Manager (part of the Executive) when requested to do so by user or kernel mode code. kernel objects are reference counted, so only when the last reference to the object is released will the object be destroyed and freed from memory.

There are quite a few object types supported by the Windows kernel. To get a peek, run the *WinObj* tool from *Sysinternals* (elevated) and locate the *ObjectTypes* directory. Figure 2-1 shows what this looks like. These types can be cataloged based on their visibility and usage:

- Types that are exported to user mode via the Windows API. Examples: mutex, semaphore, file, process, thread, timer. This book discusses many of these object types.
- Types that are not exported to user mode, but are documented in the *Windows Driver Kit* (WDK) for use by device driver writers. Examples: device, driver, callback.
- Types that are not documented even in the WDK (at least at the time of writing). These object types are for use by the kernel itself only. Examples: partition, keyed event, core messaging.

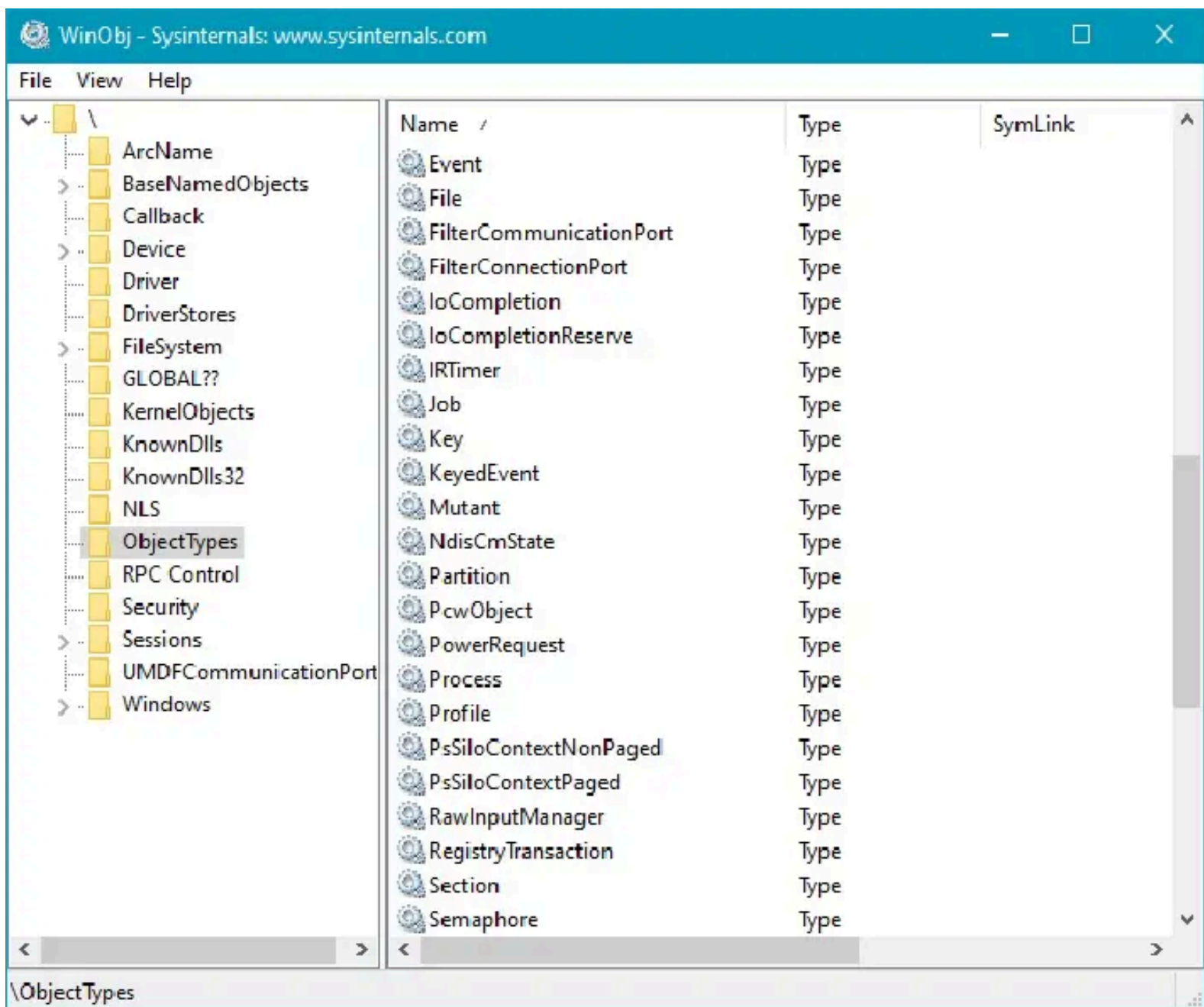


Figure 2-1: Object types

The main attributes of a kernel object are depicted in figure 2-2.

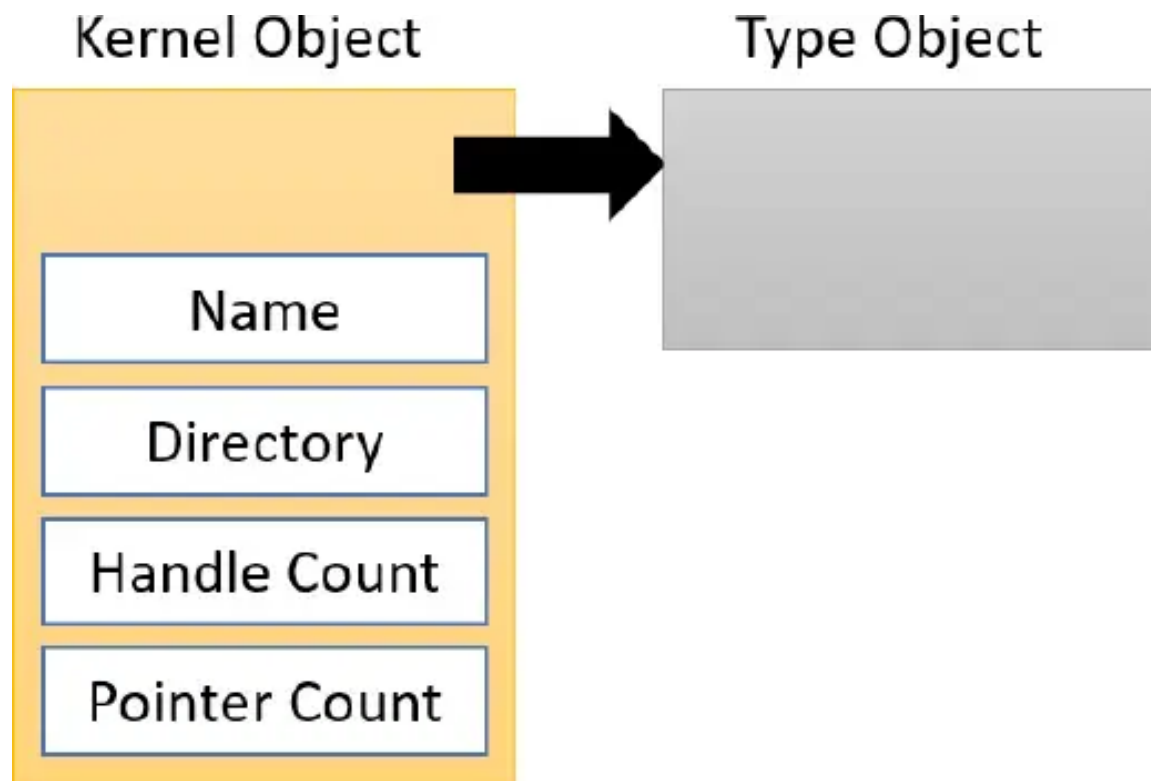


Figure 2-2: Kernel Object Attributes

Since kernel objects reside in system space, they cannot be accessed directly from user mode. Applications must use an indirect mechanism to access kernel objects, known as handles. Handles provide at least the following benefits:

- Any change in the object type's data structure in a future Windows release will not affect any client.
- Access to the object can be controlled through security access checks.
- Handles are private to a process, so having a handle to a particular object in one process means nothing in another process context.

Kernel objects are reference counted. The Object Manager maintains a handle count and a pointer count, the sum of which is the total reference count for an object (direct pointers can be obtained from kernel mode). Once an object used by a user mode client is no longer needed, the client code should close the handle used to access the object by calling `CloseHandle`. From that point on, the code should consider the handle to be invalid. Trying to access the object through the closed handle will fail, with `GetLastError` returning `ERROR_INVALID_HANDLE` (6). The client does not know, in the general case, whether the object has been destroyed or not. The Object Manager will delete the object if its reference drops to zero.

Handle values are multiples of 4, where the first valid handle is 4; Zero is never a valid handle value. This scheme does not change on 64 bit systems.

A handle is logically an index to an array of entries in a handle table maintained on a process by process basis, that points logically to a kernel object residing in system space. There are various `Create*` and `Open*` functions to create/open objects and retrieve back handles to these objects. If the object cannot be created or opened, the returned handle is in most cases `NULL` (0). One notable exception to this rule is the `CreateFile` function that returns `INVALID_HANDLE_VALUE` (-1) if it fails.

For example, the `CreateMutex` function allows creating a new mutex or opening a mutex by name (depending whether the mutex with that name exists). If successful, the function returns a handle to the mutex. A return value of zero means an invalid handle (and a function call failure). The `OpenMutex` function, on the other hand, tries to open a handle to a named mutex. If the mutex with that name does not exist, the function fails.

If the function succeeds and a name was provided, the returned handle can be to a new mutex or to an existing mutex with that name. The code can check this by calling `GetLastError` and comparing the result to `ERROR_ALREADY_EXISTS`. If it is, then it's not a new object, but rather another handle to an existing object. This is one of those rare cases where `GetLastError` can be called even if the API in question succeeded.

Running a Single Instance Process

One fairly well-known usage for the `ERROR_ALREADY_EXISTS` case is limiting an executable to have a single process instance. Normally, if you double-click an executable in Explorer, a new process is spawned based on that executable. If you repeat this operation, another process is created based on the same executable. What if you wanted to prevent the second process from launching, or at least have it shut down if it detects another process instance with the same executable already running.

The trick is using some named kernel object (a mutex is usually employed, although any named object type can be used instead), where an object with a particular name is created. If the object already exists, there must be another instance already running, so the process can shut down (possibly notifying its sibling of that fact).

The *SingleInstance* demo application demonstrates how this can be achieved. It's a dialog-based application built with WTL. Figure 2-3 shows what this application looks like running. If you try launching more instances of this application, you'll find that the first window logs messages coming from the new process instance that then exits.

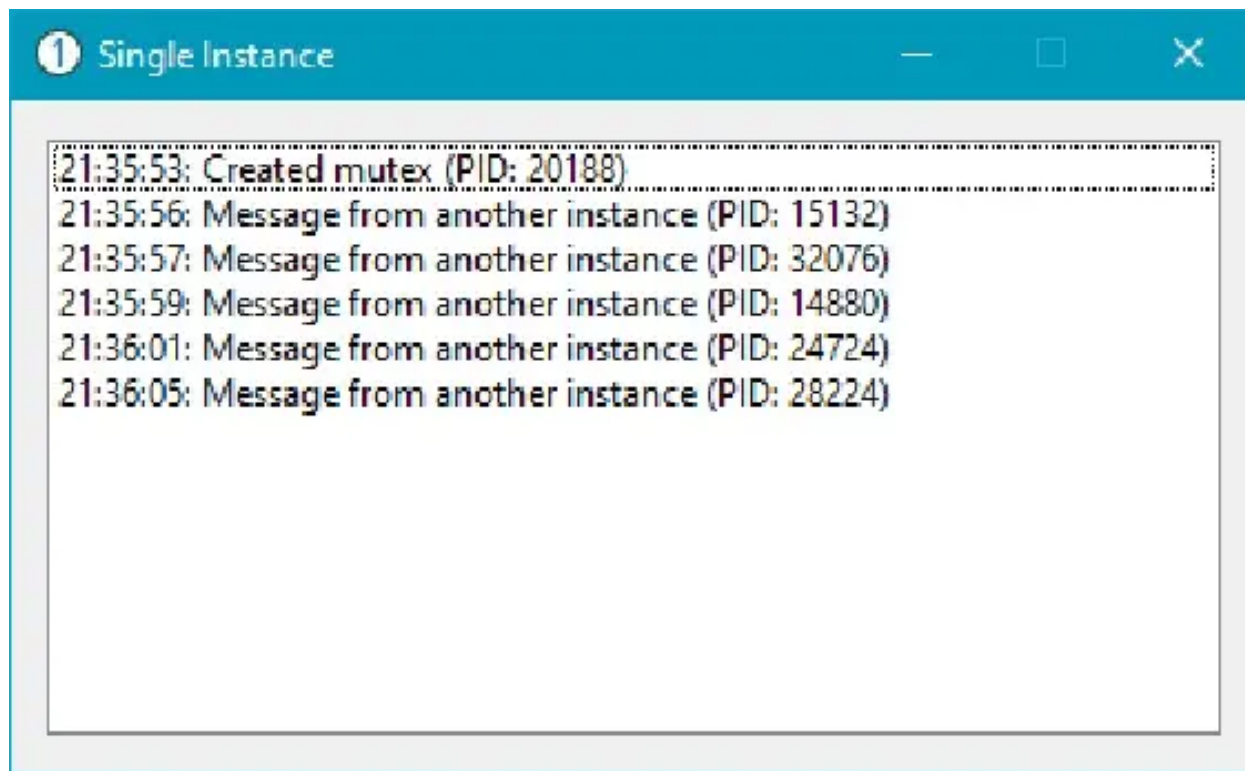


Figure 2-3: The Single Instance application

In the `WinMain` function, we create the mutex first. If this fails, then something is very wrong and we bail out.

```
HANDLE hMutex = ::CreateMutex(nullptr, FALSE, L"SingleInstanceMutex");  
if (!hMutex) {  
    CString text;  
    text.Format(L"Failed to create mutex (Error: %d)", ::GetLastError());  
    ::MessageBox(nullptr, text, L"Single Instance", MB_OK);  
    return 0;  
}
```

Failure to create the mutex should be extremely rare. The most likely scenario for failure is that another kernel object (which is not a mutex) with that same name already exists.

Now that we get a proper handle to the mutex, the only question is whether the mutex was actually created or we received another handle to an existing mutex (presumably created by a previous instance of this executable):

```

if (::GetLastError() == ERROR_ALREADY_EXISTS) {
    NotifyOtherInstance();
    return 0;
}

```

If the object existed prior to the `CreateMutex` call, then we call a helper function that sends some message to the existing instances and exits. Here is `NotifyOtherInstance`:

```

#define WM_NOTIFY_INSTANCE (WM_USER + 100)

void NotifyOtherInstance() {
    auto hWnd = ::FindWindow(nullptr, L"Single Instance");
    if (!hWnd) {
        ::MessageBox(nullptr, L"Failed to locate other instance window",
            L"Single Instance", MB_OK);
        return;
    }

    ::PostMessage(hWnd, WM_NOTIFY_INSTANCE, ::GetCurrentProcessId(), 0);
    ::ShowWindow(hWnd, SW_NORMAL);
    ::SetForegroundWindow(hWnd);
}

```

The function searches for the existing window with the `FindWindow` function and uses the window caption as the search criteria. This is not ideal in the general case, but it's good enough for this sample.

Once the window is located, we send a custom message to the window with the current process ID as an argument. This shows up in the dialog's list box.

The final piece of the puzzle is handling the `WM_NOTIFY_INSTANCE` message by the dialog. In WTL, window messages are mapped to functions using macros. The message map of the dialog class (`CMainDlg`) in *MainDlg.h* is repeated here:

```

BEGIN_MSG_MAP(CMainDlg)
    MESSAGE_HANDLER(WM_NOTIFY_INSTANCE, OnNotifyInstance)
    MESSAGE_HANDLER(WM_INITDIALOG, OnInitDialog)
    COMMAND_ID_HANDLER(IDCANCEL, OnCancel)
END_MSG_MAP()

```

The custom message is mapped to the `OnNotifyInstance` member function, implemented like so:

```

LRESULT CMainDlg::OnNotifyInstance(UINT, WPARAM wParam, LPARAM, BOOL &) {
    CString text;
    text.Format(L"Message from another instance (PID: %d)", wParam);

    AddText(text);
    return 0;
}

```

The process ID is extracted from the `wParam` parameter and some text is added to the list box with the `AddText` helper function:

```

void CMainDlg::AddText(PCWSTR text) {
    CTime dt = CTime::GetCurrentTime();
    m_List.AddString(dt.Format(L"%T") + L": " + text);
}

```

`m_List` is of type `CListBox`, a WTL wrapper for a Windows list box control.

Handles

As mentioned in the previous section, a handle points indirectly to a small data structure in kernel space that holds a few pieces of information for that handle. Figure 2-4 depicts this data structure for 32 bit and 64 bit systems.

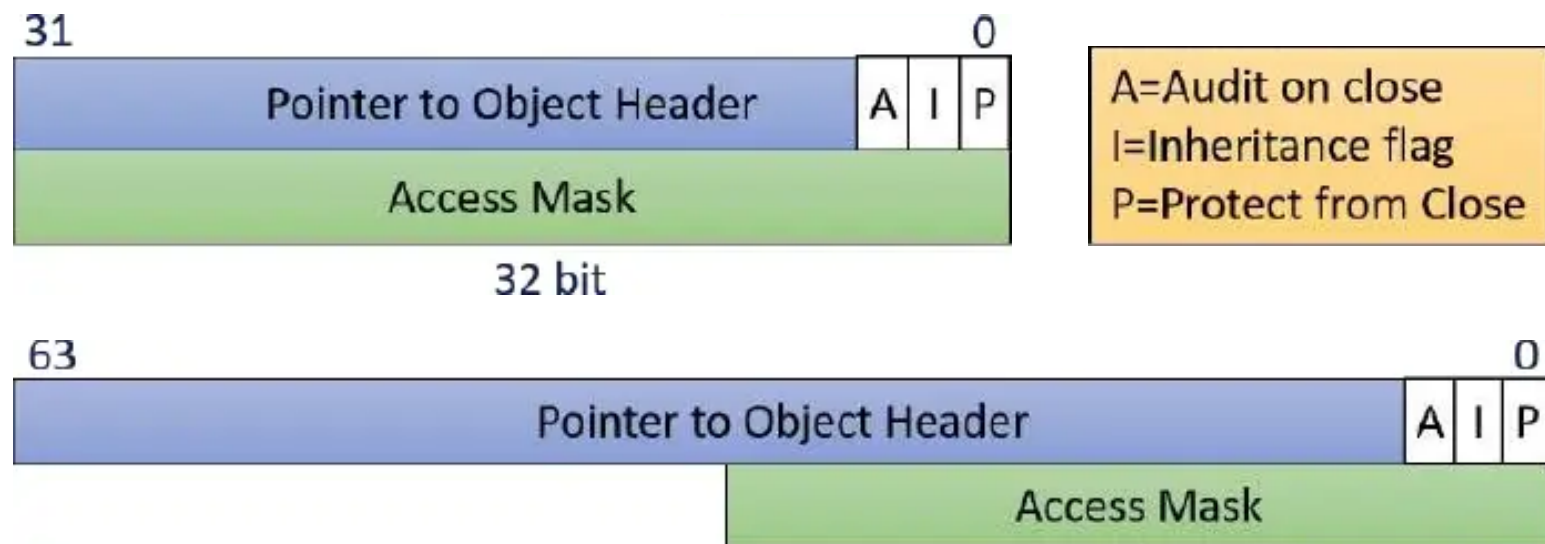


Figure 2-4: Handle Entry

On 32 bit systems, this handle entry is 8 bytes in size, and it's 16 bytes in size on 64 bit systems (technically 12 bytes are enough, but it's extended to 16 bytes for alignment purposes). Each entry has the following ingredients:

- Pointer to the actual object. Since the lower bits are used for flags and to improve CPU access times by address alignment, an object's address is multiple of 8 on 32 bit systems and multiple of 16 on 64 bit systems.
- Access mask, indicating what can be done with this handle. In other words, the access mask is the power of the handle.
- Three flags: Inheritance, Protect from close and Audit on close (discussed shortly).

The access mask is a bitmask, where each "1" bit indicating a certain operation that can be carried using that handle. The access mask is set when the handle is created by creating an object or opening an existing object. If the object is created, then the caller typically has full access to the object. But if the object is opened, the caller needs to specify the required access mask, which it may or may not get.

For example, if an application wants to terminate a certain process, it must call the `OpenProcess` function first, to obtain a handle to the required process with an access mask of (at least) `PROCESS_TERMINATE`, otherwise there is no way to terminate the process with that handle. If the call succeeds, then the call to `TerminateProcess` is bound to succeed.

Here is an example of terminating a process given its process ID:

```

bool KillProcess(DWORD pid) {
    // open a powerful-enough handle to the process
    HANDLE hProcess = ::OpenProcess(PROCESS_TERMINATE, FALSE, pid);
    if (!hProcess)
        return false;

    // now kill it with some arbitrary exit code
    BOOL success = ::TerminateProcess(hProcess, 1);

    // close the handle
    ::CloseHandle(hProcess);

    return success != FALSE;
}

```

The `OpenProcess` function has the following prototype:

```

HANDLE OpenProcess(
    _In_ DWORD dwDesiredAccess,    // the access mask
    _In_ BOOL bInheritHandle,     // inheritance flag
    _In_ DWORD dwProcessId);      // process ID

```

Since this is an *Open* operation, the object in question already exists, the client needs to specify what access mask it requires to access the object. An access mask has two types of access bits: generic and specific. We'll discuss these details in chapter 16 ("Security"). One of the specific access bits for a process is `PROCESS_TERMINATE` used in the above example. Other bits include `PROCESS_QUERY_INFORMATION`, `PROCESS_VM_OPERATION` and more. Refer to the documentation of `OpenProcess` to locate the complete list.

What access mask should be used by client code? Generally, it should reflect the operations the client code intends to perform with the object. Asking for more than needed may fail, and asking less is obviously not good enough.

The flags associated with each handle are the following:

- Inheritance - this flag is used for handle inheritance - a mechanism that allows sharing an object between cooperating processes. We'll discuss handle inheritance in chapter 3.
- Audit on close - this flag indicates whether an audit entry in the security log should be written when that handle is closed. This flag is rarely used and is off by default.

- Protect from close - setting this flag prevents the handle from being closed. A call to `CloseHandle` will return `FALSE` and `GetLastError` returns `ERROR_INVALID_HANDLE` (6). If the process is running under a debugger, an exception is raised with the following message: “0xC0000235: NtClose was called on a handle that was protected from close via `NtSetInformationObject`”. This flag is rarely useful.

Changing the inheritance and protection flags can be done with the `SetHandleInformation` function defined like so:

```
#define HANDLE_FLAG_INHERIT          0x00000001
#define HANDLE_FLAG_PROTECT_FROM_CLOSE 0x00000002
```

```
BOOL SetHandleInformation(
    _In_ HANDLE hObject,
    _In_ DWORD dwMask,
    _In_ DWORD dwFlags);
```

The first parameter is the handle itself. The second parameter is a bit mask indicating which flags to operate on. The last parameter is the actual value for these flags. For example, to set the “protect from close” bit on some handle, the following code could be used:

```
::SetHandleInformation(h, HANDLE_FLAG_PROTECT_FROM_CLOSE,
    HANDLE_FLAG_PROTECT_FROM_CLOSE);
```

Conversely, the following code snippet removes this same bit:

```
::SetHandleInformation(h, HANDLE_FLAG_PROTECT_FROM_CLOSE, 0);
```

The opposite function to read back these flags exists as well:

```
BOOL GetHandleInformation(
    _In_ HANDLE hObject,
    _Out_ LPDWORD lpdwFlags);
```

The handles opened from a particular process can be viewed with the *Process Explorer* tool from *Sysinternals*. Navigate to a process you’re interested in, and make sure the lower pane is visible (*View* menu, *Show Lower Pane*). The lower pane shows one of two views - switch

to Handle view (*View* menu, *Lower Pane View, Handles*). Figure 2-5 is a screenshot of the tool showing open handles in an *Explorer* process. The columns shown by default are *Type* and *Name* only. I added the following columns by right-clicking the header area and clicking *Select Columns: Handle, Object Address, Access* and *Decoded Access*.

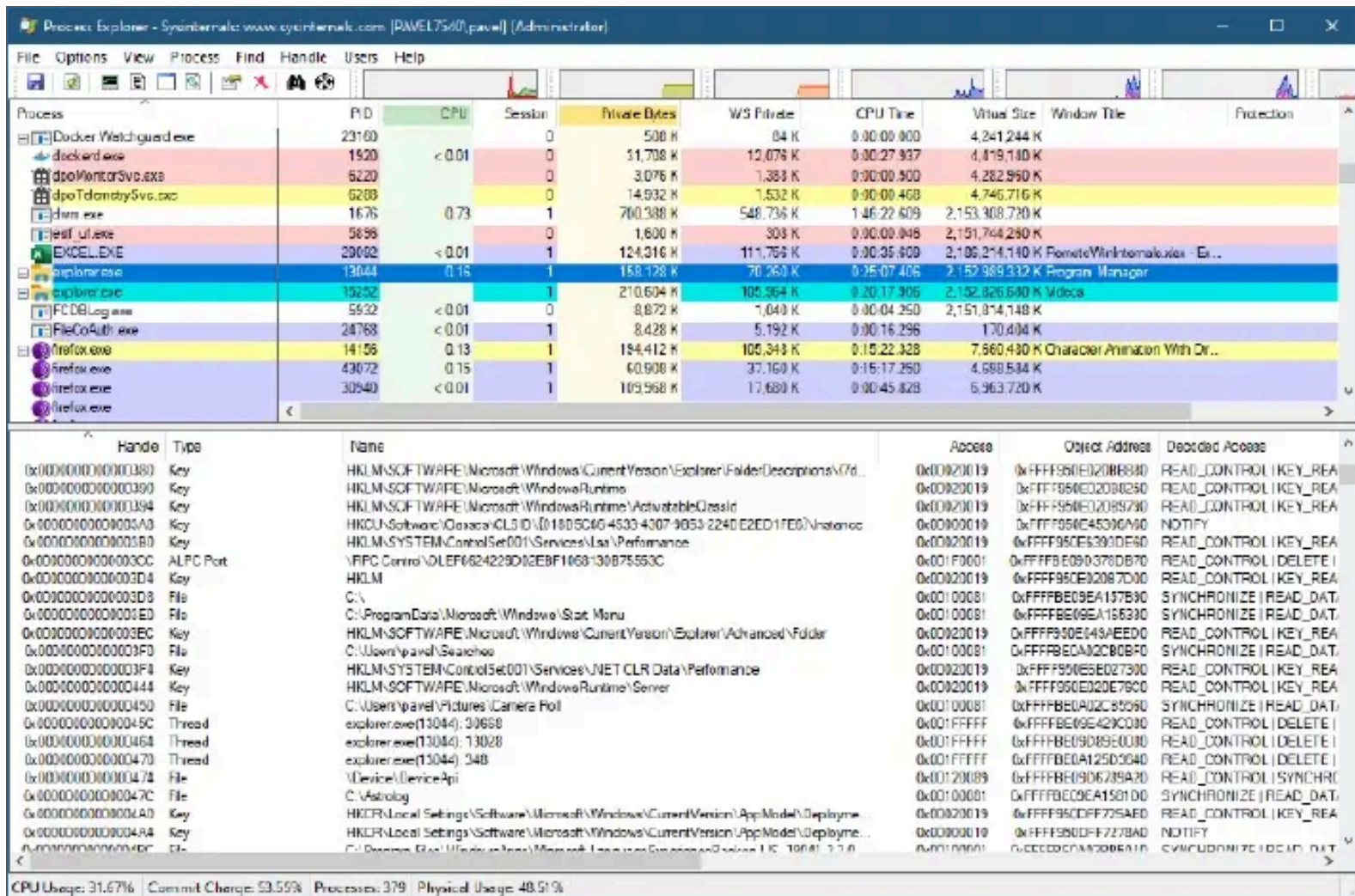


Figure 2-5: Handle View in Process Explorer

Here is a brief description of the columns:

- **Handle** - this is the handle value itself, relevant to this process only. The same handle value can have a different meaning, i.e. - points to a different object, or maybe even an empty index.
- **Type** - the object type name. This corresponds to the *Object Types* directory in *WinObj* shown in figure 2-1.
- **Object Address** - this is the kernel address where the real object structure resides. Notice these addresses end with a zero hex digit on 64 bit (on 32 bit systems, the addresses end with “8” or “0”). There is nothing user mode code with this information, but it can be used for debugging purposes: if you have two handles to an object and you want to know whether they point to the same object you can compare object

addresses; if they are the same, it's the same object. Otherwise, the handles point to different objects.

- Access - this is the access mask discussed above. To interpret the bits stored in this hex value, you need to locate the access mask bits in the documentation. To alleviate that, use the *Decoded Access* column.
- Decoded Access - provides a string representation of the access mask bits for common object types. This makes it easier to interpret the access mask bits without digging into the documentation.

I personally implemented this column to *Process Explorer*.

Process Explorer's handle view shows only handles to named objects by default. To view all handles, Enable *Show unnamed handles and mappings* option from the *View* menu. Figure 2-6 shows how the view changes when this option is checked.

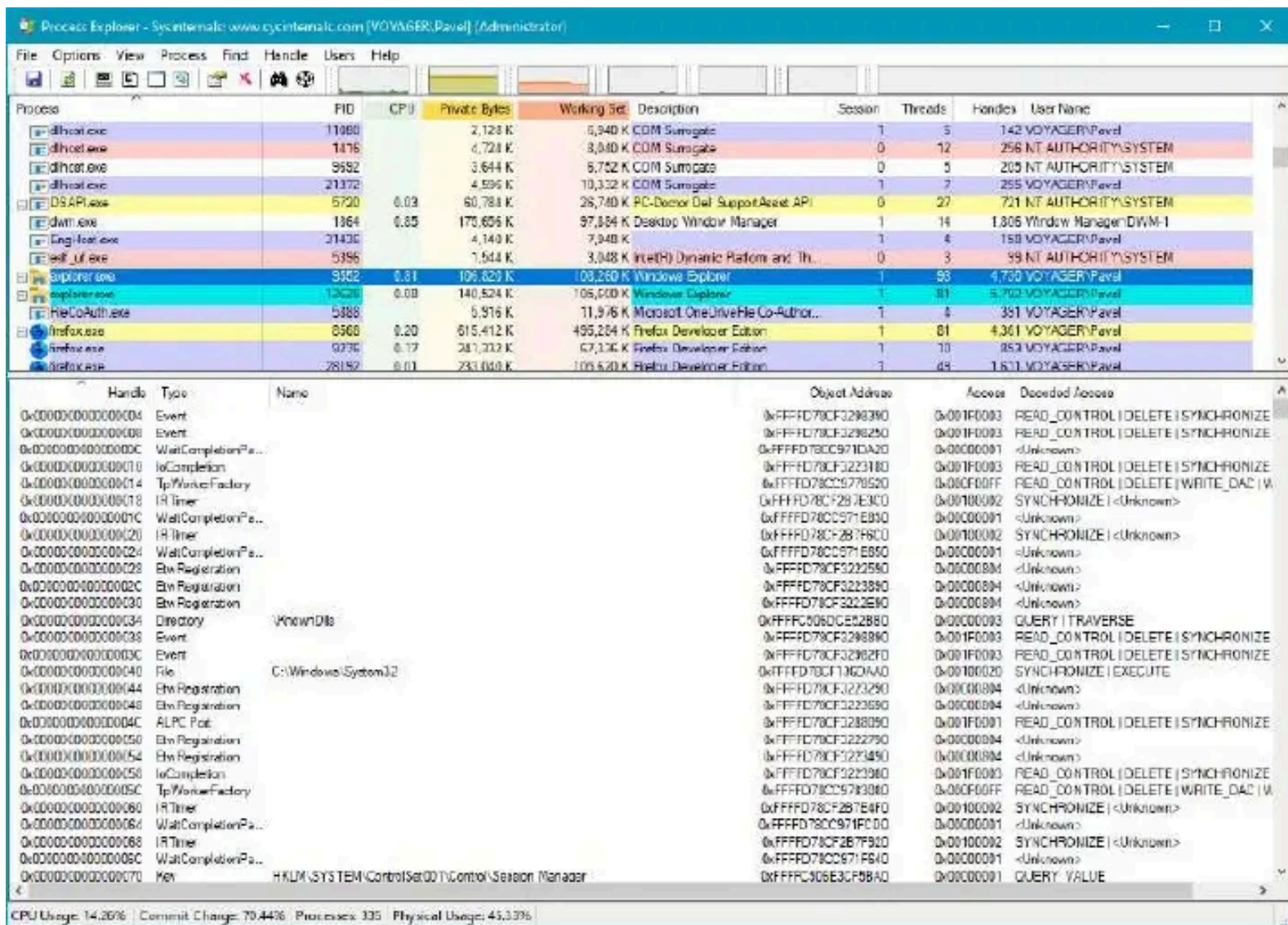
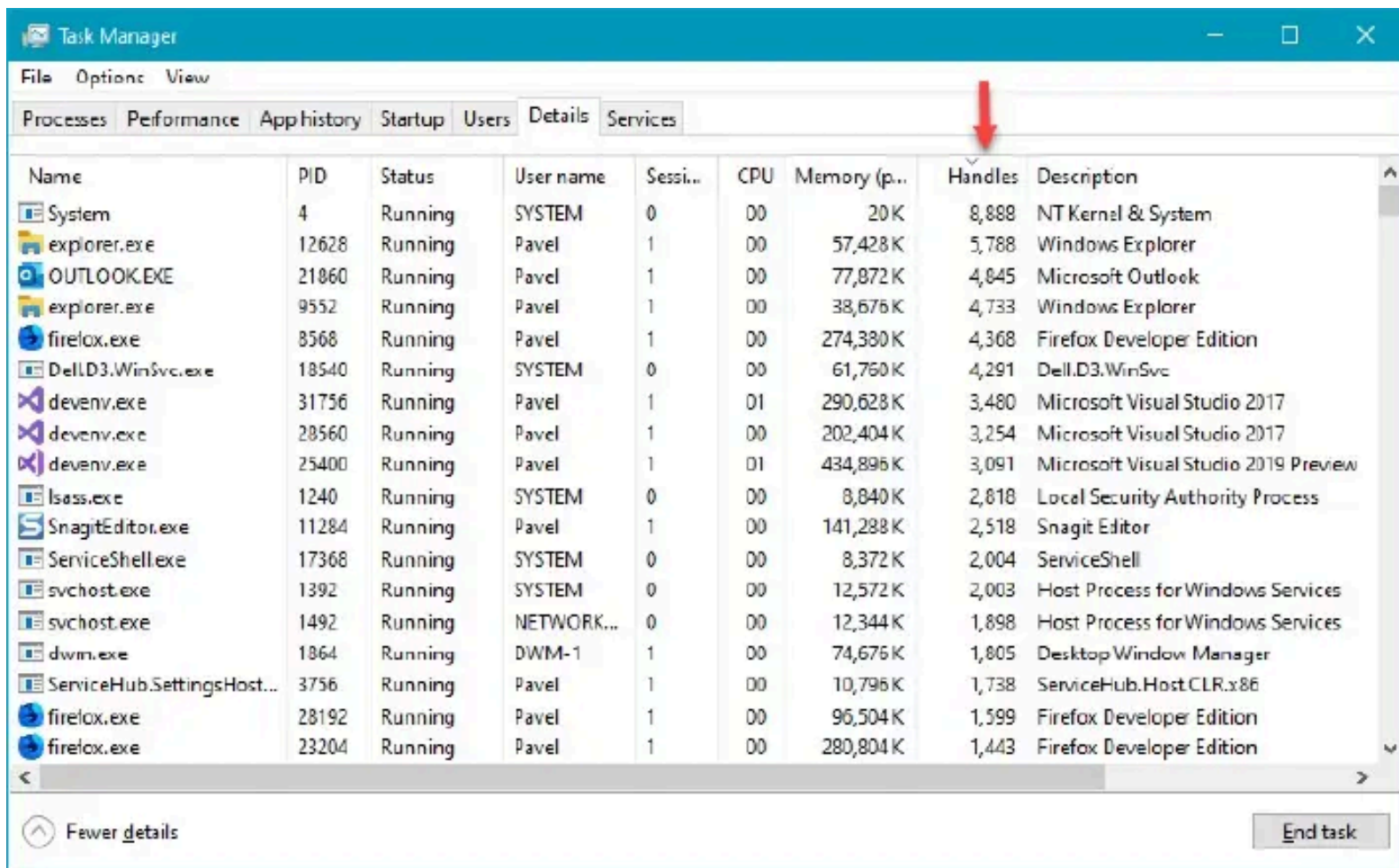


Figure 2-6: Handle View in Process Explorer (including unnamed objects)

The term “Name” is trickier than it seems. What *Process Explorer* considers named objects are not necessarily actual names, but in some cases are convenient monikers. For example, process and thread handles are shown in figure 2-5, even though processes and threads cannot have string-based names. There are other object types with a “Name” which is not their name; the most confusing are *File* and *Key*. We’ll discuss this “weirdness” in the section “Object Names”, later in this chapter.

The total number of handles in a process’ handle table is available as a column in *Process Explorer* and *Task Manager*. Figure 2-7 shows this column added to *Task Manager*.



The screenshot shows the Windows Task Manager window with the 'Details' tab selected. The 'Handles' column is highlighted, and a red arrow points to its header. The table lists various running processes with their respective handle counts.

Name	PID	Status	User name	Sessi...	CPU	Memory (p..	Handles	Description
System	4	Running	SYSTEM	0	00	20 K	8,888	NT Kernel & System
explorer.exe	12628	Running	Pavel	1	00	57,428 K	5,788	Windows Explorer
OUTLOOK.EXE	21060	Running	Pavel	1	00	77,872 K	4,845	Microsoft Outlook
explorer.exe	9552	Running	Pavel	1	00	38,676 K	4,733	Windows Explorer
firefox.exe	8568	Running	Pavel	1	00	274,380 K	4,368	Firefox Developer Edition
Dell.D3.WinSvc.exe	18540	Running	SYSTEM	0	00	61,760 K	4,291	Dell.D3.WinSvc
devenv.exe	31756	Running	Pavel	1	01	290,628 K	3,480	Microsoft Visual Studio 2017
devenv.exe	28560	Running	Pavel	1	00	202,404 K	3,254	Microsoft Visual Studio 2017
devenv.exe	25400	Running	Pavel	1	01	434,896 K	3,091	Microsoft Visual Studio 2019 Preview
lsass.exe	1240	Running	SYSTEM	0	00	8,840 K	2,818	Local Security Authority Process
SnagitEditor.exe	11284	Running	Pavel	1	00	141,288 K	2,518	Snagit Editor
ServiceShell.exe	17368	Running	SYSTEM	0	00	8,372 K	2,004	ServiceShell
svchost.exe	1392	Running	SYSTEM	0	00	12,572 K	2,003	Host Process for Windows Services
svchost.exe	1492	Running	NETWORK...	0	00	12,344 K	1,898	Host Process for Windows Services
dwm.exe	1864	Running	DWM-1	1	00	74,676 K	1,805	Desktop Window Manager
ServiceHub.SettingsHost...	3756	Running	Pavel	1	00	10,796 K	1,738	ServiceHub.Host.CLR.x86
firefox.exe	28192	Running	Pavel	1	00	96,504 K	1,599	Firefox Developer Edition
firefox.exe	23204	Running	Pavel	1	00	280,804 K	1,443	Firefox Developer Edition

Figure 2-7: Handle count column in Task Manager

Note that the number shown is the handle count, rather than object count. This is because more than one handle can exist that reference the same object.

Double-clicking a handle entry in *Process Explorer* opens a dialog that shows some properties of the object (not the handle). Figure 2-8 is a screenshot of such a dialog.

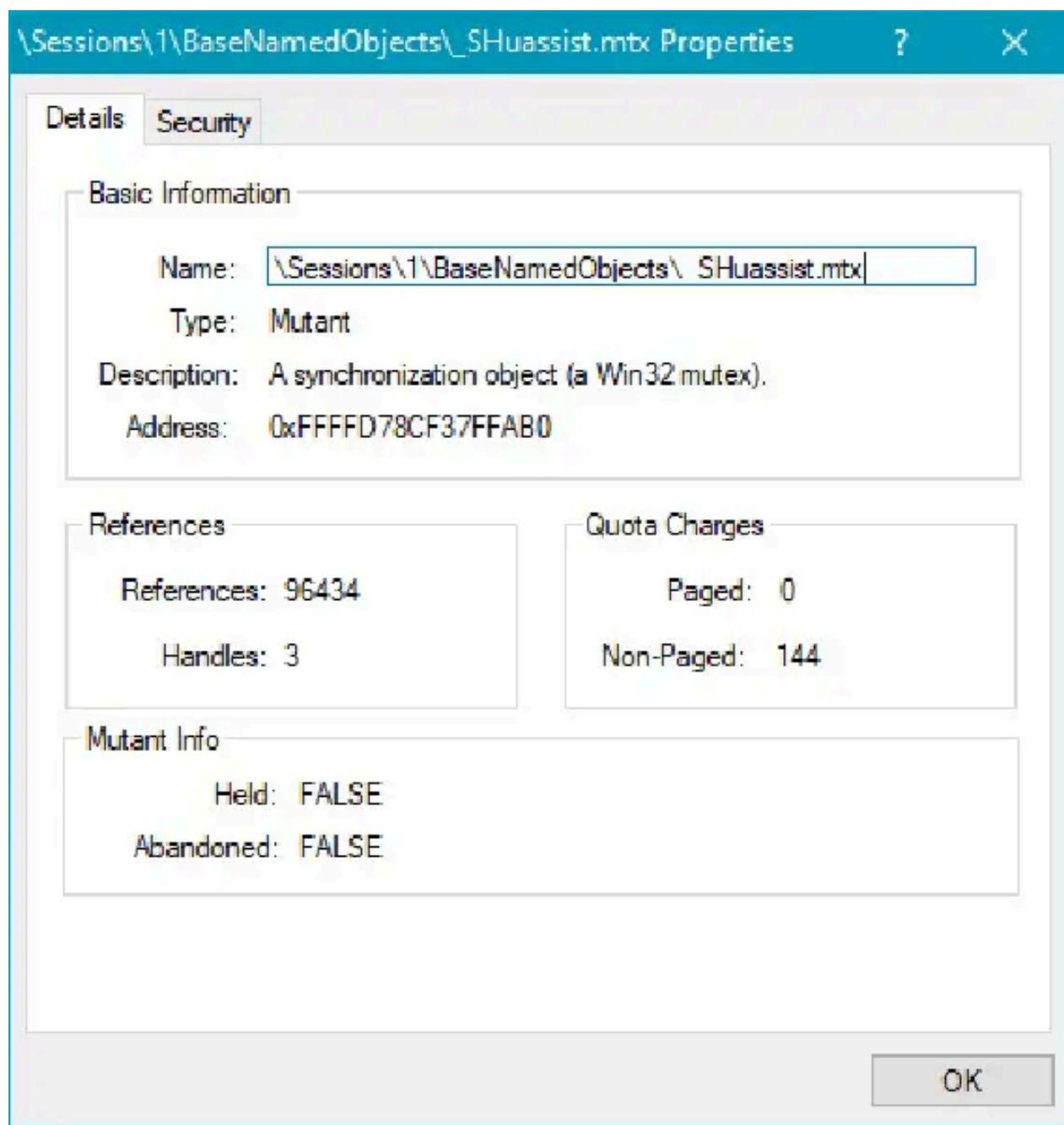


Figure 2-8: Kernel object properties in Process Explorer

The basic object information is repeated from the handle entry (name, type, and address). This particular object (a mutex) has 3 open handles. The references number is misleading and does not reflect the actual object reference count. For some types of objects (such as mutexes), extra information is shown. In this particular case, it's whether the mutex is currently held and whether it's abandoned. (we'll discuss mutexes in detail in chapter 8).

To get a sense of the number of objects and handles in the system at a given moment, you can run the *KernelObjectView* tool from my Github repository at <https://github.com/zodiacon/AllTools>. Figure 2-9 shows a screenshot of the tool. The total number of objects (per object type) is shown along with the total number of handles. You can sort by any column; which object types have the most objects? The most handles?

Name	Index	Objects	Handles	Peak Objects	Peak Handles	Pool Type	Default Paged C...	Default Non-Pag...
Token	1	12918	5306	13379	9013	Paged	0	88
Job	6	330	417	382	483	Non Paged NX	1656	0
Process	7	389	4325	1533	8739	Non Paged NX	2264	4096
Thread	8	10265	12671	13967	15273	Non Paged NX	2168	0
Partition	9	1	3	1	3	Non Paged NX	276	0
UserApocReserve	10	28	28	31	31	Non Paged NX	184	0
IoCompletionReserve	11	177	177	189	189	Non Paged NX	176	0
ActivityReference	12	0	0	3	3	Paged	0	96
PsSiloContextPaged	13	7	0	8	0	Paged	0	88
PsSiloContextNonP...	14	6	0	7	0	Non Paged NX	88	0
DebugObject	15	9	9	10	10	Non Paged NX	88	0
Event	16	63499	65340	75402	79041	Non Paged NX	112	0
Mutant	17	6858	7978	7589	8712	Non Paged NX	144	0
Callback	18	53	0	53	1	Non Paged NX	88	0
Semaphore	19	12593	12965	15072	16384	Non Paged NX	120	0
Timer	20	582	621	687	727	Non Paged NX	476	0
IRTimer	21	2496	2504	5949	6957	Non Paged NX	256	0
Profile	22	0	0	0	0	Non Paged NX	328	0
KeyedEvent	23	1	0	1	1	Paged	0	88
WindowStation	24	8	802	10	2832	Non Paged NX	208	0
Desktop	25	13	374	17	1378	Non Paged NX	344	0
Composition	26	1138	420	1217	443	Non Paged NX	24	0
RawInputManager	27	35	59	40	63	Non Paged NX	904	0
CoreMessaging	28	16	16	16	16	Non Paged NX	104	0
ActivationObject	29	0	0	0	0	Paged Session NX	0	90
TpWorkerFactory	30	1237	1237	3463	3463	Non Paged NX	664	0

Figure 2-9: Kernel object Viewer

Pseudo Handles

Some handles have special values and are not closable. These are known as *pseudo-handles*, although they are used just like any other handle when needed. Calling `CloseHandle` on pseudo-handles always fails. Here are the functions returning pseudo-handles:

- `GetCurrentProcess` (-1) - return a pseudo-handle to the calling process
- `GetCurrentThread` (-2) - return a pseudo-handle to the calling thread
- `GetCurrentProcessToken` (-4) - return a pseudo-handle to the token of the calling process
- `GetCurrentThreadToken` (-5) - return a pseudo-handle to the token of the calling thread
- `GetCurrentThreadEffectiveToken` (-6) - return a pseudo-handle to the effective token of the calling thread (if the thread has its own token - it's used, otherwise - its process token is used)

The last three pseudo handles (token handles) are only supported on Windows 8 and later, and their access mask is `TOKEN_QUERY` and `TOKEN_QUERY_SOURCE` only.

Processes, threads and tokens are discussed later in this book.

RAII for Handles

It's important to close a handle once it's no longer needed. Applications that fail to do that properly may exhibit "handle leak", where the number of handles grows uncontrollably if the application opens handles but "forgets" to close them. Obviously, this is bad.

One way to help code manage handles without forgetting to close them is to use C++ by implementing a well-known idiom called *Resource Acquisition is Initialization* (RAII). The name is not that good, but the idiom is. The idea is to use a destructor for a handle wrapped in a type that ensures the handle is closed when that wrapper object is destroyed.

Here is a simple RAII wrapper for a handle (implemented inline for convenience):

```
struct Handle {
    explicit Handle(HANDLE h = nullptr) : _h(h) {}
    ~Handle() { Close(); }

    // delete copy-ctor and copy-assignment
    Handle(const Handle&) = delete;
    Handle& operator=(const Handle&) = delete;

    // allow move (transfer ownership)
    Handle(Handle&& other) : _h(other._h) {
        other._h = nullptr;
    }
    Handle& operator=(Handle&& other) {
        if (this != &other) {
            Close();
            _h = other._h;
            other._h = nullptr;
        }
    }
}
```

```

        return *this;
    }

    operator bool() const {
        return _h != nullptr && _h != INVALID_HANDLE_VALUE;
    }

    HANDLE Get() const {
        return _h;
    }

    void Close() {
        if (_h) {
            ::CloseHandle(_h);
            _h = nullptr;
        }
    }

private:
    HANDLE _h;
};

```

The `Handle` type provides the basic operations expected from a RAII `HANDLE` wrapper. The copy constructor and copy assignment operators are removed, as copying a handle that may have multiple owners does not make sense (causing `CloseHandle` to be called twice for the same handle). It is possible to implement these copy operations by duplicating the handle (see “Sharing Kernel Objects” later in this chapter), but it’s a non-trivial operation best avoided in implicit copy scenarios. A `bool` operator returns true if the current handle held is valid; it considers zero and `INVALID_HANDLE_VALUE` (-1) as invalid handles. The `Close` function closes the handle and is normally called from the destructor. Finally, the `Get` function returns the underlying handle.

It’s possible to add an implicit conversion operator to `HANDLE`, removing the need to call `Get`.

Here is some example code using the above wrapper:

```
Handle hMyEvent(::CreateEvent(nullptr, TRUE, FALSE, nullptr));
if (!hMyEvent) {
    // handle failure
    return;
}
::SetEvent(hMyEvent.Get());

// move ownership
Handle hOtherEvent(std::move(hMyEvent));
::ResetEvent(hOtherEvent.Get());
```

Although writing such a RAII wrapper is possible, it's usually best to use an existing library that provides this (and other similar) functionality. For example, although `CloseHandle` is the most common closing handle function, there are other types of handles that require a different closing function. One such library that is used by Microsoft in Windows code is the *Windows Implementation Library* (WIL). This library has been released on Github and is available as a *Nuget* package.

Using WIL

Adding WIL to a project is done like any other *Nuget* package. Right-click the *References* node in a Visual Studio project and select *Manage Nuget Packages....* In the *Browse* tab's search text box, type "wil" to quickly search for WIL. The full name of the package is "Microsoft.Windows.ImplementationLibrary", shown in figure 2-10.

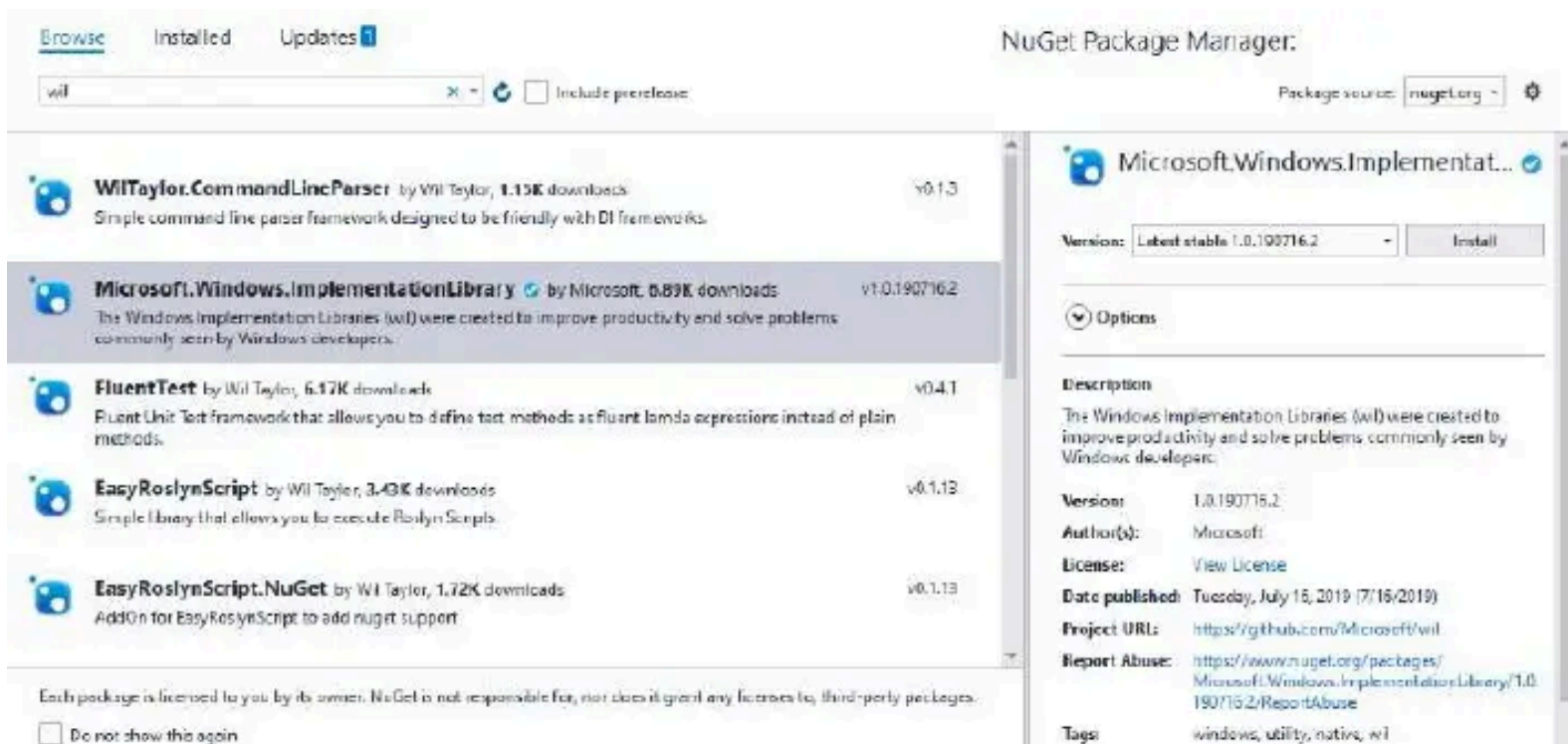


Figure 2-10: Adding WIL via NuGet

The RAII handle wrapper(s) are located in the `<wil\resource.h>` header file.

Here is the same code using WIL:

```
#include <wil\resource.h>

void DoWork() {
    wil::unique_handle hMyEvent(::CreateEvent(nullptr, TRUE, FALSE, nullptr));
    if (!hMyEvent) {
        // handle failure
        return;
    }
    ::SetEvent(hMyEvent.get());

    // move ownership
    auto hOtherEvent(std::move(hMyEvent));
    ::ResetEvent(hOtherEvent.get());
}
```

`wil::unique_handle` is a `HANDLE` wrapper that calls `CloseHandle` upon destruction. It's modeled mostly after the C++ `std::unique_ptr<>` type. Notice that getting the internal `HANDLE` is done by calling `get()`. To replace the value inside a `unique_handle` (and close the old one) use the `reset` function; calling `reset` with no arguments just closes the underlying handle, making the wrapper object an empty shell.



The code can be somewhat simplified by adding `using namespace wil;` so that `wil::` need not be prepended for every type in WIL. Also, notice `auto` can be used to simplify code in some cases.

The code samples in this book use WIL in some cases, but not all. From a learning perspective, it's sometimes better to use the raw types to make things simpler to understand.

Creating Objects

All the functions for creating new objects have some common parameters. Here is the `CreateMutex` and `CreateEvent` functions to demonstrate:

```
HANDLE CreateMutex(
    _In_opt_ LPSECURITY_ATTRIBUTES lpMutexAttributes,
    _In_ BOOL bInitialOwner,
    _In_opt_ LPCTSTR lpName);
```

```
HANDLE CreateEvent(
    _In_opt_ LPSECURITY_ATTRIBUTES lpEventAttributes,
    _In_ BOOL bManualReset,
    _In_ BOOL bInitialState,
    _In_opt_ LPCTSTR lpName);
```

Notice both functions accept a parameter of type `SECURITY_ATTRIBUTES`. This structure is common to virtually all *Create* functions is defined like so:

```
typedef struct _SECURITY_ATTRIBUTES {
    DWORD nLength;
    LPVOID lpSecurityDescriptor;
    BOOL bInheritHandle;
} SECURITY_ATTRIBUTES, *PSECURITY_ATTRIBUTES;
```

The `nLength` member should be set to the size of the structure. This is a common technique used by Windows to version structures. If the structure will have new members in a future Windows release, old code would still function properly because it would have set the length to the old size, so the newer Windows API know not to look at the new members as the old code had no idea these existed. That said, the `SECURITY_ATTRIBUTES` structure has yet to change from the first Windows NT release.

As its name implies, the structure has to do with security settings on the newly created object. The main member that is really about security is `lpSecurityDescriptor`, which can point to a security descriptor object, which essentially specifies who-can-do-what with the object. We'll discuss security descriptors in chapter 16.

The last member, `bInheritHandle` has security implications, which is why it's hosted in this structure as well. It's the inheritance bit mentioned earlier. This means the inheritance bit can be set by using this structure without needing to call `SetHandleInformation` when creating a new object. Here is an example for creating an event object with the returned handle having its inheritance bit set:

```
SECURITY_ATTRIBUTES sa = { sizeof(sa) };    // set nLength and zero the rest
sa.bInheritHandle = TRUE;

HANDLE hEvent = ::CreateEvent(&sa, TRUE, FALSE, nullptr);
DWORD flags;
::GetHandleInformation(hEvent, &flags);      // sets flags=1
```

Handle inheritance is discussed in chapter 3.

Passing `NULL` for `SECURITY_ATTRIBUTES` leaves the inheritance bit clear. In terms of security, this means “default security” which is based on a security descriptor that is stored in the process access token. We'll discuss the details in chapter 16. In any case, using `NULL` for the security descriptor (either explicitly or passing `NULL` for the `SECURITY_ATTRIBUTES` pointer) is the right thing to do in most cases.

Object Names

Some types of objects can have string-based names. These names can be used to open objects by name with a suitable *Open* function. Note that not all objects have names; for example, processes and threads don't have names - they have IDs. That's why the *OpenProcess* and *OpenThread* functions require a process/thread identifier (a number) rather than a string-based name. Named objects can be viewed with the *WinObj* tool from *Sysinternals*.

From user mode code, calling a *Create* function with a name creates the object with that name if an object with that name does not exist, but if it exists it just opens the existing object. In the latter case, calling *GetLastError* returns *ERROR_ALREADY_EXISTS*, indicating this is not a new object, and the returned handle is yet another handle to an existing object. In this case the parameters that affect object creation such as the *SECURITY_ATTRIBUTES* structure are not used, as the creator already set this up.

The name provided to a *Create* function is not the final name of the object. In classic (desktop) processes, it's prepended with *\Sessions\x\BaseNamedObjects* where *x* is the session ID of the caller. If the session is zero, the name is prepended with *\BaseNamedObjects* only. If the caller happens to be running in an AppContainer (typically a Universal Windows Platform process), then the prepended string is more complex and consists of the unique AppContainer SID: *\Sessions\x\AppDataContainerNamedObjects*.

Figure 2-11 shows named objects in session 1 in *WinObj*.

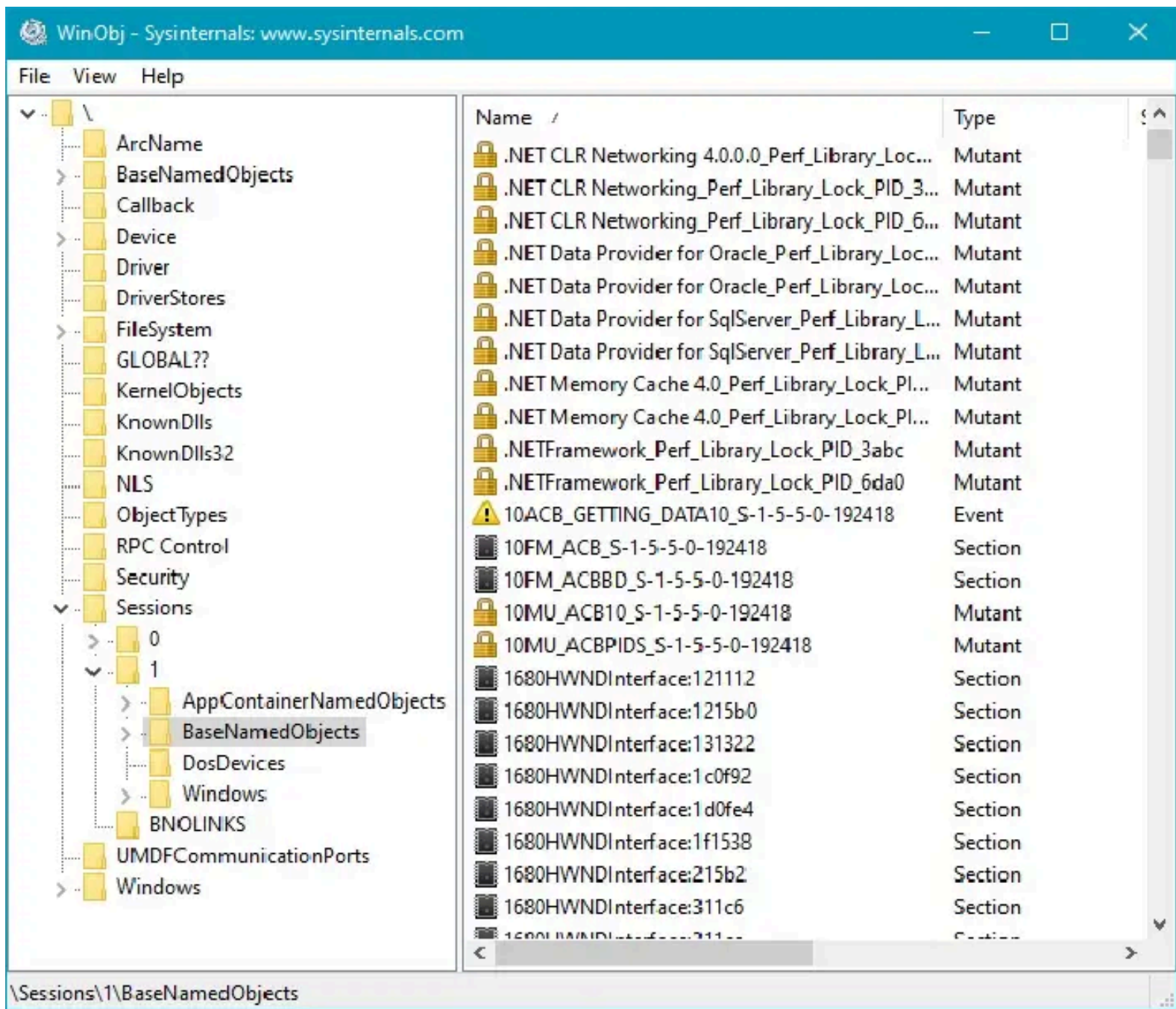


Figure 2-11: Named objects in session 1

Figure 2-12 shows named objects in session 0.

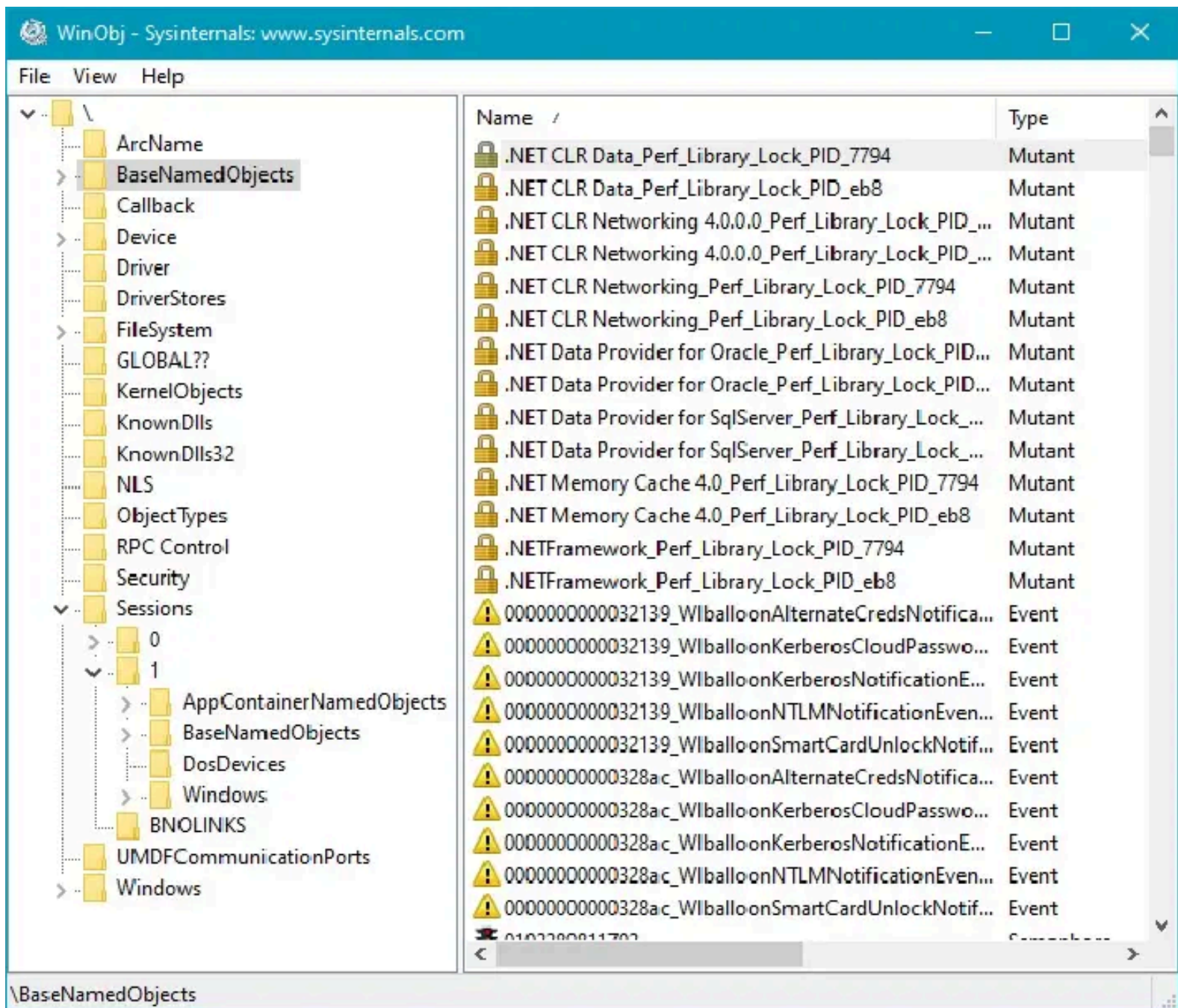


Figure 2-12: Named objects in session 0

All the above means is that object names are session-relative (and in the case of an AppContainer - package relative). If an object must be shared across sessions it can be created in session 0 by prepending the object name with *Global*; for example, creating a mutex with the `CreateMutex` function named *Global\MyMutex* will create it under *\BaseNamedObjects*. Note that AppContainers do not have the power to use session 0 object namespace.

The entire Object Manager namespace hierarchy can be viewed with *WinObj*. This entire structure is held in memory and manipulated by the Object Manager as needed. Note that unnamed objects are not part of this structure, meaning the objects seen in *WinObj* do not comprise all the existing objects, but rather all the objects that were created with a name.

The “directories” shown in *WinObj* actually Directory objects, which are just one kind of kernel object which acts as a logical container.

Going back to *Process Explorer*’s Handles view - it shows by default “named” objects. “Named” here means not just objects that can be named, but also other objects. Objects that can be named are Mutexes (Mutants), Semaphores, Events, Sections, ALPC Ports, Jobs, Timers, and other, less used object types. Yet others are shown with a name that has a different meaning than a true named object:

- Process and Thread objects - the name is shown as their unique ID.
- For File objects it shows the file name (or device name) pointed to by the file object. It’s not the same as an object’s name, as there is no way to get a handle to a file object given the file name - only a new file object may be created that accesses the same underlying file or device (assuming sharing settings for the original file object allow it).
- (Registry) Key object names are shown with the path to the registry key. This is not a name, for the same reasoning as for file objects.
- Directory objects show its logical path, rather than being a true object name. A Directory is not a file system object, but rather an object manager directory.
- Token object names are shown with the user name stored in the token.

To verify the above statements, browse through *WinObj* and look for File or Key objects. You won’t find any, which suggests these objects cannot be named.

Sharing Kernel Objects

As we’ve seen, handles to kernel objects are private to a process. In some cases, a process may want to share a kernel object with another process. Such a process cannot simply pass somehow the value of the handle to the other process, because in the other process’ handle table that handle value may point to a different object or be empty.

Clearly, some mechanism must be in place to allow such sharing. In fact, there are three:

- Sharing by name
- Sharing by handle inheritance
- Sharing by duplicating handles

We'll look at the first and third option here, and discuss handle inheritance in the next chapter.

Sharing by Name

This is the simplest option, if available. “Available” here means that the object in question can have a name, and does have a name. The typical scenario is that the cooperating processes (2 or more) would call the appropriate *Create* function with the same object name. The first process to make the call would create the object, and subsequent calls from the other processes would open additional handles to the same object.

The sample *BasicSharing* shows an example of using sharing by name with a *Memory Mapped File* object. This object can be used to share memory between processes (normally, each process can only see its own address space). Running two instances (or more) of the application (shown in figure 2-13) allows sharing textual data between these processes.

The full details of memory-mapped files are discussed in chapter 14.

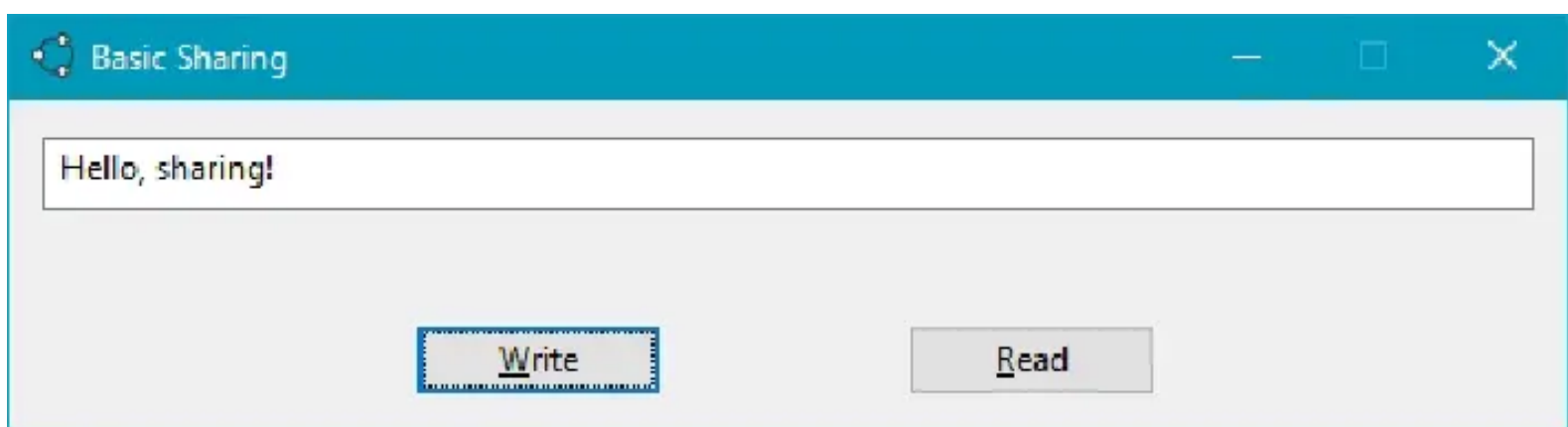


Figure 2-13: The Basic Sharing application

To test it out, type something in the edit box and click *Write*. Then switch to another instance, and just click *Read*. The text you entered should appear in the other application's edit box. Of course you can swap roles. If you launch another instance, you can click *Read* and the

last text would appear as well. This is because all these processes are reading and writing to the same (shared) memory.



By the way, these don't have to be processes based on the same executable - this is just used here for convenience. The determining factor is the object's name.

Before we look at the code, let's see what this looks like in *Process Explorer*. Run two instances of the executable, open *Process Explorer* and locate the two processes. Make sure the lower pane shows Handles (and not DLLs). The object type to look for is *Section* (the kernel name of Memory Mapped File). Find a section called "MySharedMemory" (with the session-based prefix of course), as shown in figure 2-14.

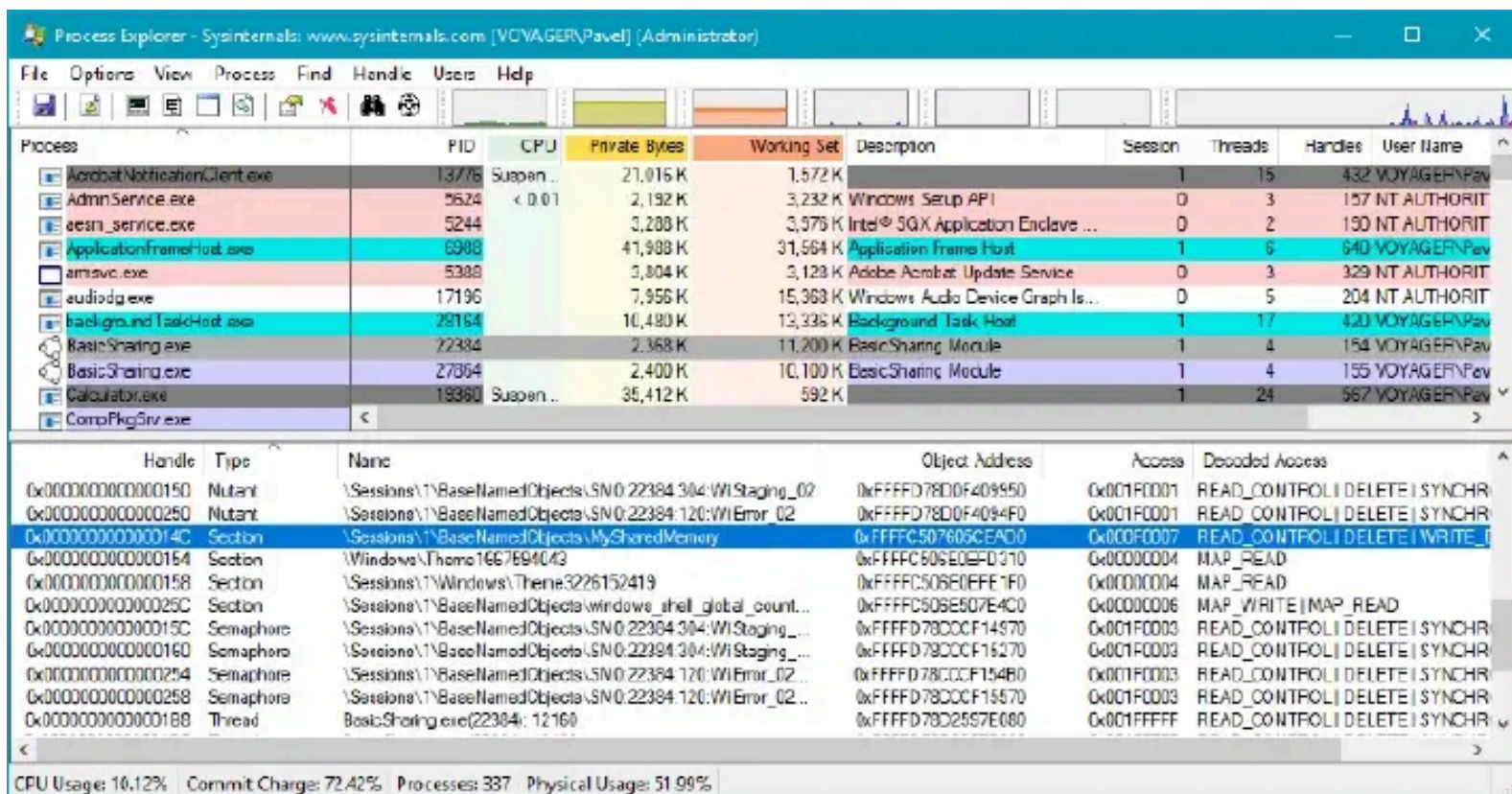


Figure 2-14: The shared section object

If you double-click the handle, you should see the properties of the section object as shown in figure 2-15.

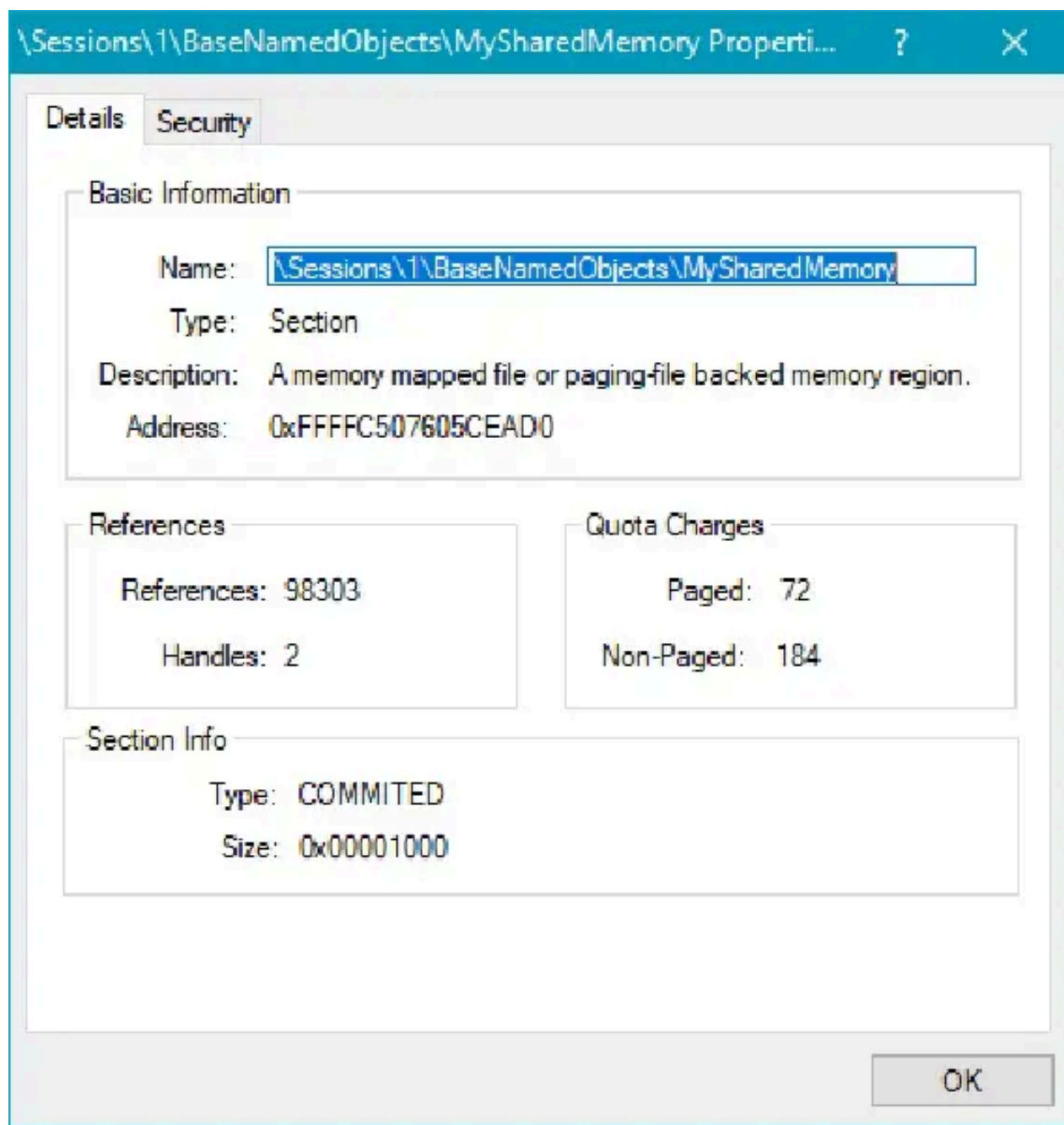


Figure 2-15: Section object properties

Notice there are two open handles to the object. Presumably, these are coming from the two processes holding handles to that object. Notice the shared memory's size: 4 KB - we'll see this reflected in the code.

If you locate the second process using this object (see figure 2-16), you should find the same information presented when double-clicking the handle. How can you be sure these are pointing to the same object? Look at the *Object Address* column. If the address is identical, this is the same object (and vice versa). Notice also that the handle values are not the same (the normal case). In figures 2-14 the handle value is 0x14c (PID 22384) and in figure 2-16 it's 0x16c (PID 27864). Still - they reference the exact same object.

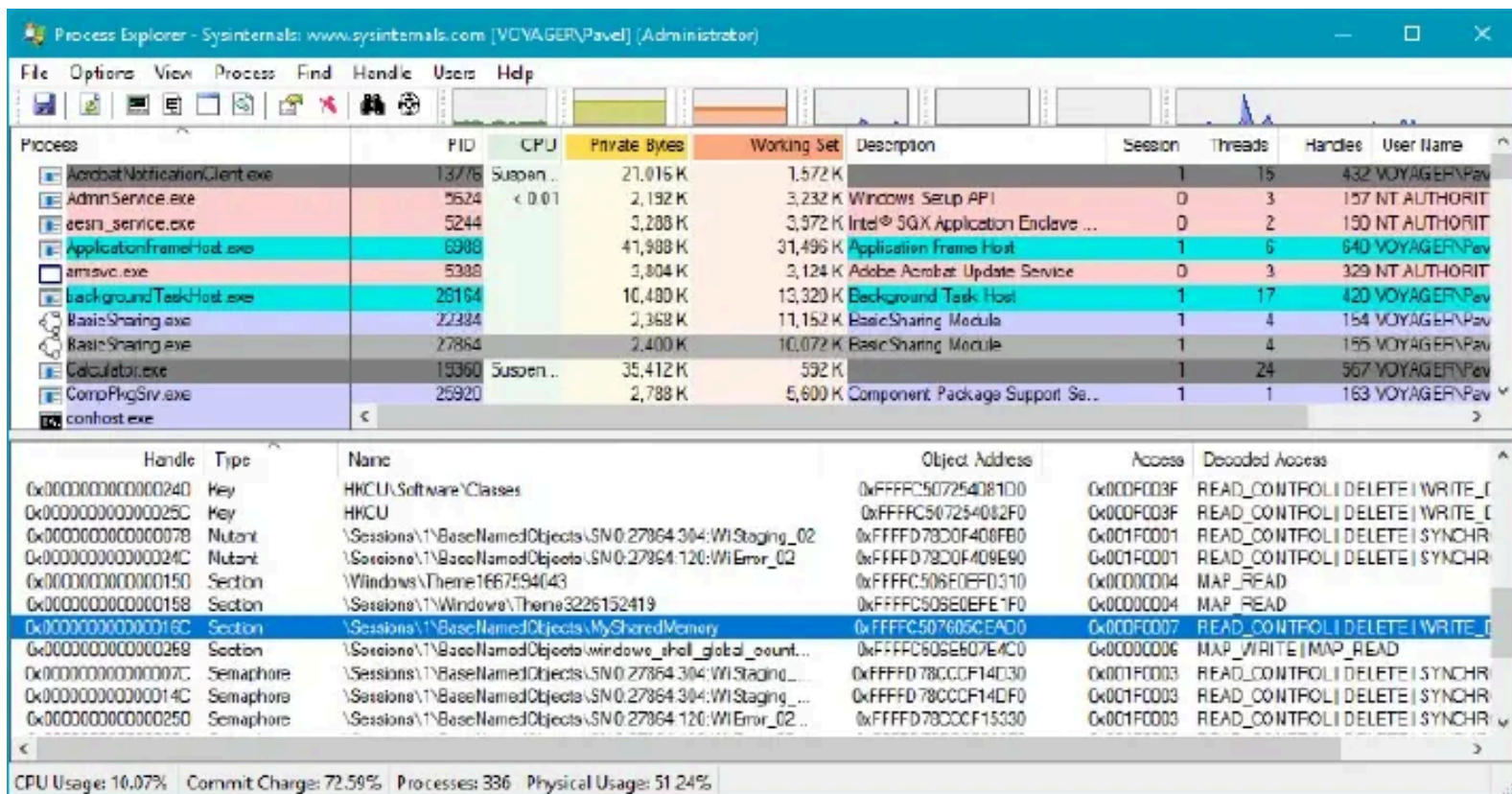


Figure 2-16: The shared section in the other process

If you were to close one of the instances, what would happen? One handle would close, but the object remains alive. This means that launching a completely new instance and clicking *Read* will show the most recent text. What would happen if we close all cooperating applications and then launch one instance again. What would we see if we click *Read*? Try and explain to yourself why this is the case.

Now let's turn our attention to the code.

BasicApplication is a WTL dialog-based project. The dialog box class (*CMainDlg*) holds a single member of interest, which is the handle to the memory-mapped file:

private:

```
HANDLE m_hSharedMem;
```

When the dialog is created, in the *WM_INITDIALOG* message handler, we create the file mapping object and giving it a name:

```

m_hSharedMem = ::CreateFileMapping(INVALID_HANDLE_VALUE, nullptr, PAGE_READWRITE,
    0, 1 << 12, L"MySharedMemory");
if (!m_hSharedMem) {
    AtlMessageBox(m_hWnd, L"Failed to create/open shared memory", IDR_MAINFRAME\
);
    EndDialog(IDCANCEL);
}

```

CreateFileMapping is used to create (or open) a file mapping object. The exact details of the parameters are discussed in chapter 14 (in part 2). Here we care about one parameter in particular (the last) - the object's name. This is the name we've seen in *Process Explorer* (with the standard session-related prefix). If this is the first process to attempt creating the object - it's created. Subsequent calls result in additional handles to the same object (calling GetLastError would return ERROR_ALREADY_EXISTS). In this case, we don't care whether this call is the first or not - we just want a handle to the same kernel object so that its "function" is available from multiple processes.

The second to last argument pair (0 and $1 \ll 12$) determine the size of the shared memory as a 64-bit value. In this case it's set to 4 KB ($1 \ll 12$). If the call fails for any reason we just print a simple message and close the dialog, causing the process itself to exit.

When the dialog is closed, it's a good idea to close the handle. Strictly speaking, it's not necessary to do that in this particular case, because once the dialog is closed, the process exits, and the kernel ensures that all handles from a terminated process are properly closed. Still, it's a good habit to have (unless some RAII wrapper for the handle does that for you). For completeness, here is the call to close handle when handling the WM_DESTROY message for the dialog:

```

if (m_hSharedMem)
    ::CloseHandle(m_hSharedMem);

```

Now for the write and read parts. Accessing the shared memory is done by calling MapViewOfFile, resulting in a pointer to the shared memory (again, the exact details are in chapter 12). Then it's just a matter of copying the text to that mapped memory:

```

void* buffer = ::MapViewOfFile(m_hSharedMem, FILE_MAP_WRITE, 0, 0, 0);
if (!buffer) {
    AtlMessageBox(m_hWnd, L"Failed to map memory", IDR_MAINFRAME);
    return 0;
}

CString text;
GetDlgItemText(IDC_TEXT, text);
::wcscpy_s((PWSTR)buffer, text.GetLength() + 1, text);

::UnmapViewOfFile(buffer);

```

The copying is done with `wcscpy_s` to the mapped memory. Then the memory is unmapped with `UnmapViewOfFile`.

Reading data is very similar. The access mask is changed to `FILE_MAP_READ` rather than `FILE_MAP_WRITE`, and memory is copied in the other direction, directly to the edit box:

```

void* buffer = ::MapViewOfFile(m_hSharedMem, FILE_MAP_READ, 0, 0, 0);
if (!buffer) {
    AtlMessageBox(m_hWnd, L"Failed to map memory", IDR_MAINFRAME);
    return 0;
}

SetDlgItemText(IDC_TEXT, (PCWSTR)buffer);
::UnmapViewOfFile(buffer);

```

Sharing by Handle Duplication

Sharing kernel objects by name is certainly simple. What about objects that don't (or can't have a name)? Handle duplication may be the answer. Handle duplication has no inherent restrictions (except security) - it can work on almost any kernel object, named or unnamed and it works at any point in time (in chapter 3 we'll see that handle inheritance is only available when a process creates a child process). There is a dent, however; this is the most difficult way of sharing in practice, as we shall soon see.



A Duplicated I/O completion port handle does not work in the target process.

Duplicating a handle is as simple as calling the `DuplicateHandle` function:

```
BOOL DuplicateHandle(  
    _In_ HANDLE hSourceProcessHandle,  
    _In_ HANDLE hSourceHandle,  
    _In_ HANDLE hTargetProcessHandle,  
    _Outptr_ LPHANDLE lpTargetHandle,  
    _In_ DWORD dwDesiredAccess,  
    _In_ BOOL bInheritHandle,  
    _In_ DWORD dwOptions);
```

Duplicating a handle requires a source process, source handle and a target process. If successful, a new handle entry is written to the target process handle table, pointing to the same object as the source handle. The “before” and “after” duplication are depicted in figures 2-17 and 2-18, respectively.

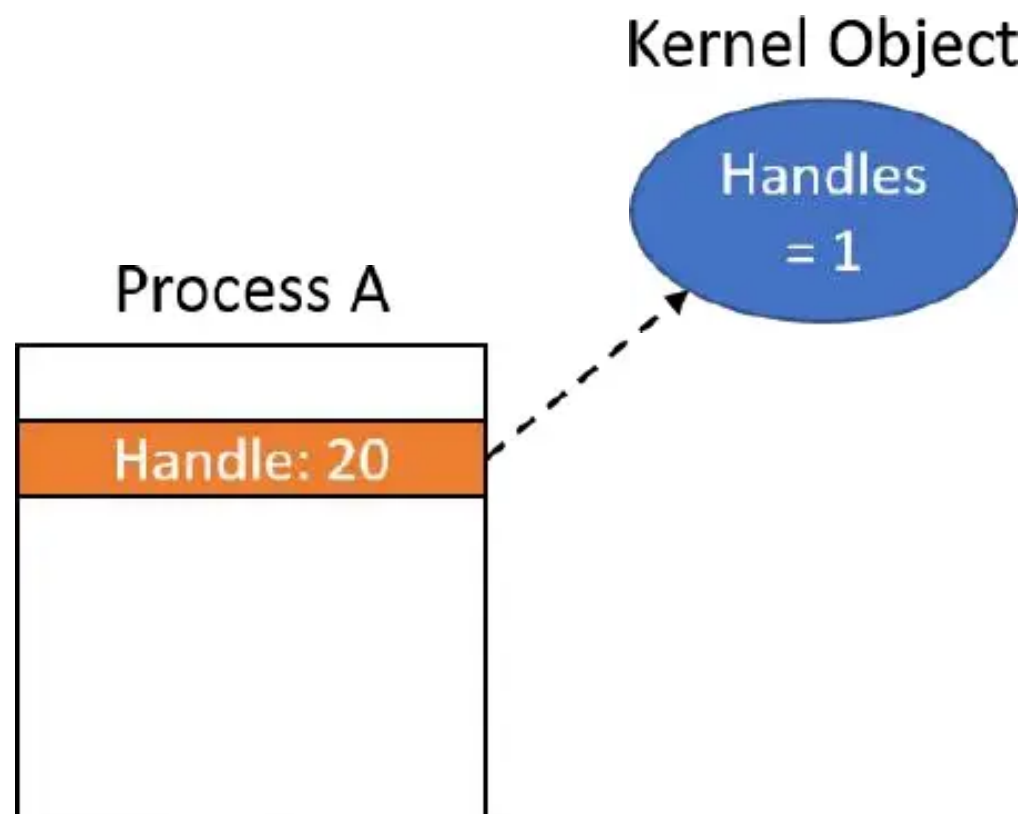


Figure 2-17: Before handle duplication

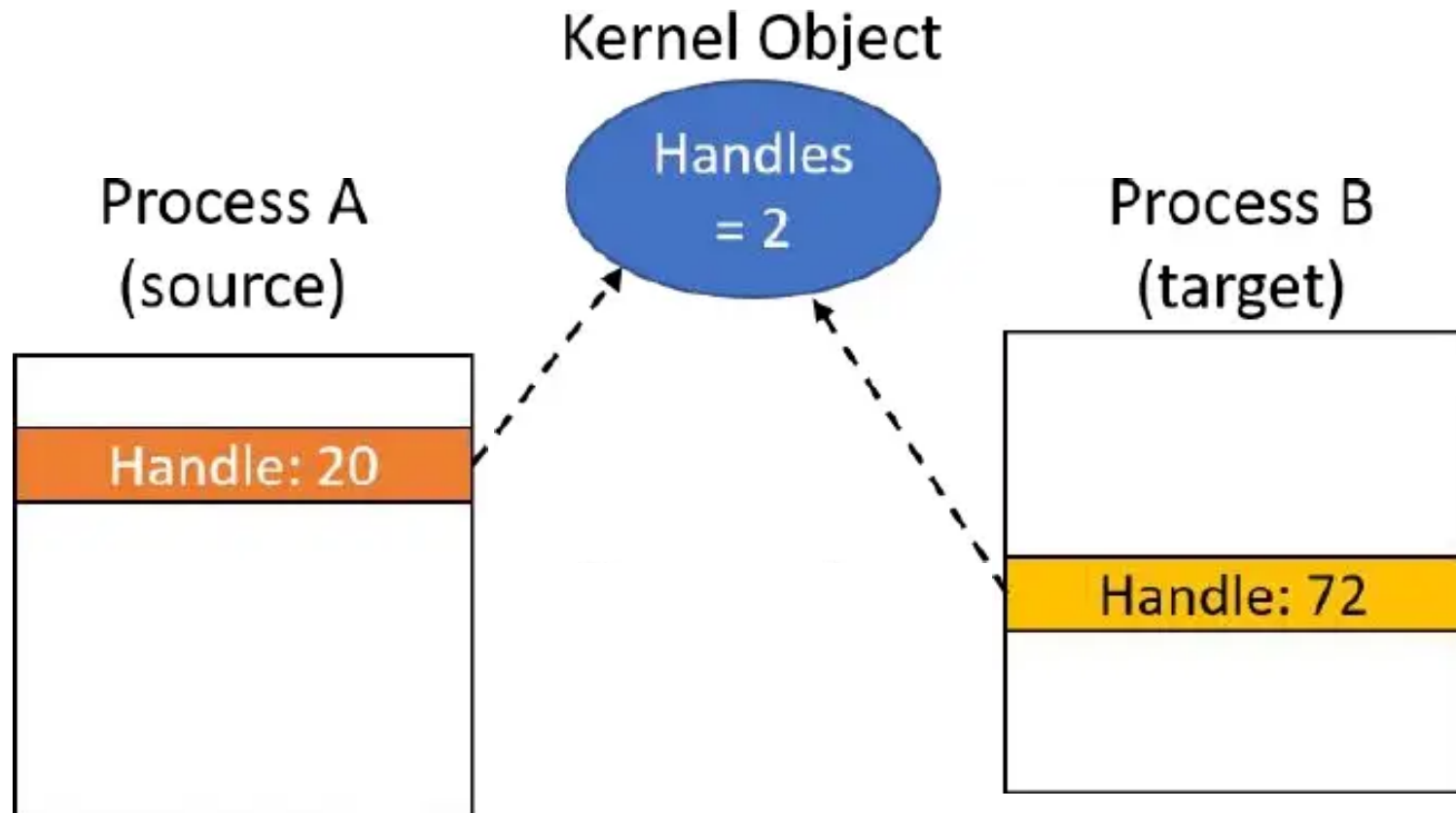


Figure 2-18: After handle duplication

Technically, `DuplicateHandle` can work on any two processes for which proper handles can be obtained, but the typical scenario is duplication one of the caller's handles into another process' handle table. Also, the source and target processes may be the same. Let's go over the parameters of `DuplicateHandle` in details:

- `hSourceProcessHandle` - this is a handle to the source process. This handle must have the `PROCESS_DUP_HANDLE` access mask. If the source is the caller's process than passing `GetCurrentProcess` will do the trick (and it always has full access).
- `hSourceHandle` - the source handle to duplicate. This handle must be valid in the context of the source process.
- `hTargetProcessHandle` - the target process handle. Typically some call to `OpenProcess` must be used to gain such a handle. As with the source process, the `PROCESS_DUP_HANDLE` access mask is required.
- `lpTargetHandle` - this is the resulting handle, **valid from the target process perspective**. In figure 2-18, the resulting handle returned to the caller was 72. This value is with respect to Process B (the caller is assumed to be process A).
- `dwDesiredAccess` - the desired access mask for the duplicated handle. If the `dwOptions` parameter has the flag `DUPLICATE_SAME_ACCESS`, then this access mask is ignored. Otherwise, this is the access mask to request for the new handle.

- `bInheritHandle` - specifies whether the new handle is inheritable or not (see chapter 3 for more on handle inheritance).
- `dwOptions` - a set of flags. One is `DUPLICATE_SAME_ACCESS` discussed above. The second supported is `DUPLICATE_CLOSE_SOURCE`; if specified, closes the source handle after a successful duplication (this means the handle count for the object is not incremented).

Here is a simple example of creating a job object and duplicating a handle to it in the same process while reducing the access mask (error handling omitted):

```
HANDLE hJob = ::CreateJobObject(nullptr, nullptr);
HANDLE hJob2;
::DuplicateHandle(::GetCurrentProcess(), hJob, ::GetCurrentProcess(), &hJob2,
    JOB_OBJECT_ASSIGN_PROCESS | JOB_OBJECT_TERMINATE, FALSE, 0);
```

The source and target process are the current process. Running this piece of code and looking at the handles in *Process Explorer* shows the differences (figure 2-19).

Handle	Type	Name	Access	Decoded Access
0x000000000000008C	IoCompletion		0x001F0003	READ_CONTROL DELETE SYNCHRONIZE WRITE_DAC WRITE_OWNER <Unknown>
0x000000000000001C	IRTimer		0x00100002	SYNCHRONIZE <Unknown>
0x0000000000000024	IRTimer		0x00100002	SYNCHRONIZE <Unknown>
0x0000000000000090	IRTimer		0x00100002	SYNCHRONIZE <Unknown>
0x000000000000009C	IRTimer		0x00100002	SYNCHRONIZE <Unknown>
0x00000000000000AC	Job		0x001F003F	READ_CONTROL DELETE SYNCHRONIZE WRITE_DAC WRITE_OWNER JOB_OBJECT_ALL_ACCESS
0x00000000000000B0	Job		0x00000009	ASSIGN_PROCESS TERMINATE
0x0000000000000008	Key	HKLM\SOFTWARE\...	0x00000009	QUERY_VALUE ENUMERATE_SUB_KEYS

Figure 2-19: Simple handle duplication

One handle (0xac) has full access to the job object, while the other (duplicated) handle (0xb0) has just the specified desired access mask.

In the more common case, a handle from the current process is duplicated to a target cooperating process. The following function will duplicate a source handle from the current process to a target process:

```
HANDLE DuplicateToProcess(HANDLE hSource, DWORD pid) {  
    // open a strong-enough handle to the target process  
    HANDLE hProcess = ::OpenProcess(PROCESS_DUP_HANDLE, FALSE, pid);  
    if (!hProcess)  
        return nullptr;  
  
    HANDLE hTarget = nullptr;  
  
    // duplicate  
    ::DuplicateHandle(::GetCurrentProcess(), hSource, hProcess,  
        &hTarget, 0, FALSE, DUPLICATE_SAME_ACCESS);  
  
    // cleanup  
    ::CloseHandle(hProcess);  
  
    return hTarget;  
}
```

This is the case where handle duplication becomes non-trivial. It's not the act of duplication itself - that's rather simple - a single function call. The problem is how to convey the information to the target process. Two pieces of information must be conveyed to the target process:

- When the handle has been duplicated.
- What is the duplicated handle value?

Remember, that the caller knows the created handle value, but the target process does not. There must be some other form of inter-process communication that allows the caller process to pass the required information to the target process (since they are part of the same system and need to cooperate by sharing the kernel object in question).

We'll look at various inter-process communication mechanisms throughout this book.

Private Object Namespaces

We've seen that some types of kernel objects can have string-based names. We've also seen that this is one (convenient) way of sharing such objects between processes. However, there are a few downsides of having named objects:

- Some other, unrelated process, may create an object with the same name, that can cause failure when creating the object later (if the object types differ), or worse, the creation “succeeds” because it's the same object type and the code gets back a handle to an existing object. The result is a mess, where processes use the same object that they don't expect.
- This is a special case of the above bullet, for emphasis. Since the name is visible (in tools, but can also be obtained programmatically), another process can “hijack” the object or otherwise interfere with object usage. From a security standpoint, the object in question is too visible. Unnamed objects are much stealthier, as there is no good way to guess what a particular object is used for.

Is there a way for processes to share named objects (since it's easy) but not be visible to other processes? Starting with Windows Vista, there is a way to create a private object namespace that only the cooperating processes know about. Using tools or APIs will not reveal its full name.

The *PrivateSharing* sample application is an enhanced version of *BasicSharing*, where the memory-mapped file object's name is now under a private object namespace and is not visible to all. Looking at this object with *Process Explorer* shows a partial name only (figure 2-20).

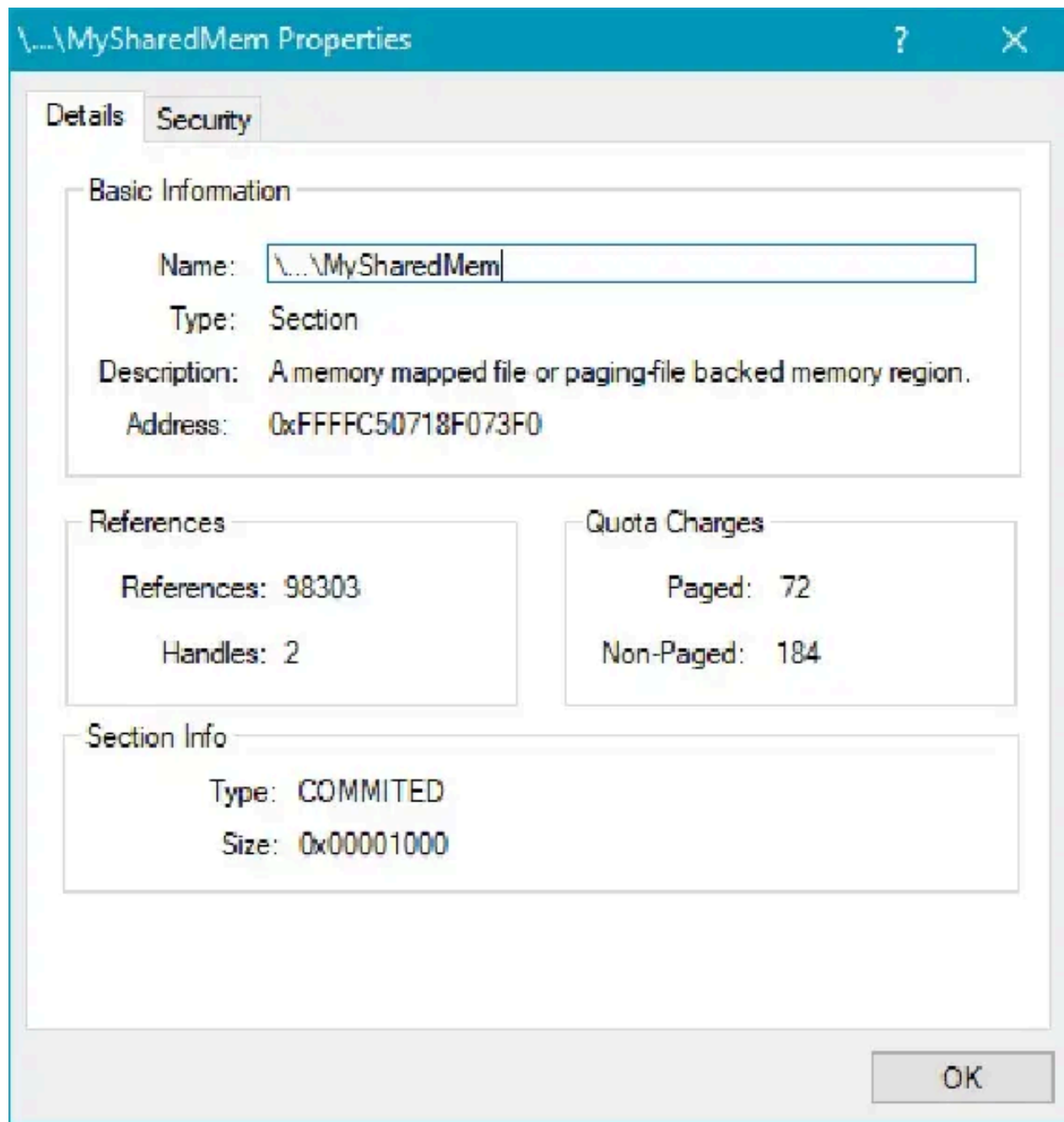


Figure 2-20: Named object with a private namespace

If some random code tries to locate an object named “MySharedMem”, it would fail to do so, since this not the object’s true name.

Creating a private namespace is a two-step process. First, a helper object called a *Boundary Descriptor* must be created. This descriptor allows adding certain Security IDs (SIDs) that would be able to use private namespaces created based on that boundary descriptor. This can help tighten security on the private namespace(s). To create a boundary descriptor, use `CreateBoundaryDescriptor`:

```
HANDLE CreateBoundaryDescriptor(  
    _In_ LPCTSTR Name,  
    _In_ ULONG Flags); // currently unused
```

Once a boundary descriptor exists, two functions can be used to restrict access to any private namespace created through that descriptor: `AddSIDToBoundaryDescriptor` and `AddIntegrityLabelToBoundaryDescriptor` (the latter available starting from Windows 7):

```
BOOL AddSIDToBoundaryDescriptor(  
    _Inout_ HANDLE* BoundaryDescriptor,  
    _In_ PSID RequiredSid);  
  
BOOL AddIntegrityLabelToBoundaryDescriptor(  
    _Inout_ HANDLE * BoundaryDescriptor,  
    _In_ PSID IntegrityLabel);
```

Both accept the address of the boundary descriptor's handle and a SID. With `AddSIDToBoundaryDescriptor`, the SID is typically a group's SID, allowing all users in that group access to the private namespaces. `AddIntegrityLabelToBoundaryDescriptor` allows setting a minimum integrity level for processes that wish to open objects in private namespace managed by this boundary descriptor.

SIDs and integrity levels are discussed in chapter 16.

Once the boundary descriptor is set, the next step is creating the actual private namespace with `CreatePrivateNamespace`:

```
HANDLE CreatePrivateNamespace(  
    _In_opt_ LPSECURITY_ATTRIBUTES lpPrivateNamespaceAttributes,  
    _In_ LPVOID lpBoundaryDescriptor, // the boundary descriptor  
    _In_ LPCWSTR lpAliasPrefix); // namespace name
```

Confusingly, the boundary descriptor type is `void*` rather than `HANDLE`. This is a slip in the API, but since `HANDLE` is defined as `void*`, this works fine. This mishap also hints that a

boundary descriptor is not a kernel object, even though it returns a HANDLE; it has its own close function - `DeleteBoundaryDescriptor`.

An object namespace is also not a true kernel object. If the namespace already exists, the function fails and `OpenPrivateNamespace` must be used instead. It also has its own close function (`ClosePrivateNamespace`):

```
HANDLE OpenPrivateNamespaceW(
    _In_ LPVOID lpBoundaryDescriptor,
    _In_ LPCWSTR lpAliasPrefix);    // namespace name

BOOLEAN ClosePrivateNamespace(
    _In_ HANDLE Handle,
    _In_ ULONG Flags);            // 0 or PRIVATE_NAMESPACE_FLAG_DESTROY
```

Another slip is the function `ClosePrivateNamespace` returning `BOOLEAN` (typedefed as `BYTE`) instead of the standard `BOOL`.

Once the namespace is created or opened, named objects can be created normally with the name in the form `alias\name` where “alias” is the `lpAliasPrefix` parameter from creating or opening the namespace.

Let’s look at the concrete code in the *PrivateSharing* application.

The dialog class now has three members:

```
private:
    wil::unique_handle m_hSharedMem;
    HANDLE m_hBD{ nullptr }, m_hNamespace{ nullptr };
```

The code uses the `WIL unique_handle` RAII wrapper for the memory-mapped file’s handle, but the boundary descriptor and the namespace are managed as raw handles.

When the dialog box is created, the same memory-mapped file is created as in *BasicSharing*, but this time under a private namespace (error handling omitted for clarity):

```

// create the boundary descriptor
m_hBD = ::CreateBoundaryDescriptor(L"MyDescriptor", 0);

BYTE sid[SECURITY_MAX_SID_SIZE];
auto psid = reinterpret_cast<PSID>(sid);
DWORD sidLen;
::CreateWellKnownSid(WinBuiltinUsersSid, nullptr, psid, &sidLen);

::AddSIDToBoundaryDescriptor(&m_hBD, psid);

// create the private namespace
m_hNamespace = ::CreatePrivateNamespace(nullptr, m_hBD, L"MyNamespace");
if (!m_hNamespace) {    // maybe created already?
    m_hNamespace = ::OpenPrivateNamespace(m_hBD, L"MyNamespace");
}

m_hSharedMem.reset(::CreateFileMapping(INVALID_HANDLE_VALUE, nullptr,
    PAGE_READWRITE, 0, 1 << 12, L"MyNamespace\\MySharedMem"));

```

In this example, a single SID was added to the boundary descriptor. This SID is for all standard users. It's possible to add something more strict, such as the Administrators group, so that processes running under standard user rights would not be able to tap into this boundary descriptor. The SID is created based on a well-known SID for the users group by calling `CreateWellKnownSid`. Then `AddSIDToBoundaryDescriptor` is called to attach the SID to the boundary descriptor.

Don't worry about these SIDs and other security terms. They are described in detail in chapter 16.

Once the boundary descriptor is set, `CreatePrivateNamespace` or `OpenPrivateNamespace` is called with the alias "MyNamespace". This is used as the prefix for the memory-mapped file object created with `CreateFileMapping`.

Finally, the `WM_DESTROY` message handler for the dialog deletes the namespace and boundary descriptor:

```

if (m_hNamespace)
    ::ClosePrivateNamespace(m_hNamespace, 0);
if (m_hBD)
    ::DeleteBoundaryDescriptor(m_hBD);

```

Bonus: WIL Wrappers for Private Namespaces

The WIL library has many wrappers for various types of handles and pointers. Unfortunately, it doesn't have a boundary descriptor and private namespace wrappers. Fortunately, it's not too difficult to create ones. Here is one way to do it:

```

namespace wil {
    static void close_private_ns(HANDLE h) {
        ::ClosePrivateNamespace(h, 0);
    };

    using unique_private_ns = unique_any_handle_null_only<decltype(
        &close_private_ns), close_private_ns>;

    using unique_bound_desc = unique_any_handle_null_only<decltype(
        &::DeleteBoundaryDescriptor), ::DeleteBoundaryDescriptor>;
}

```

I will not go over the details of the above declarations, since they do require good acquaintance with C++ 11 `decltype`, `using` and templates.

The *PrivateSharing2* project is the same as *PrivateSharing* but uses WIL wrappers (with the above additions) to manage all handles and even the pointer returned from `MapViewOfFile`. Here is the *Read* function for example:

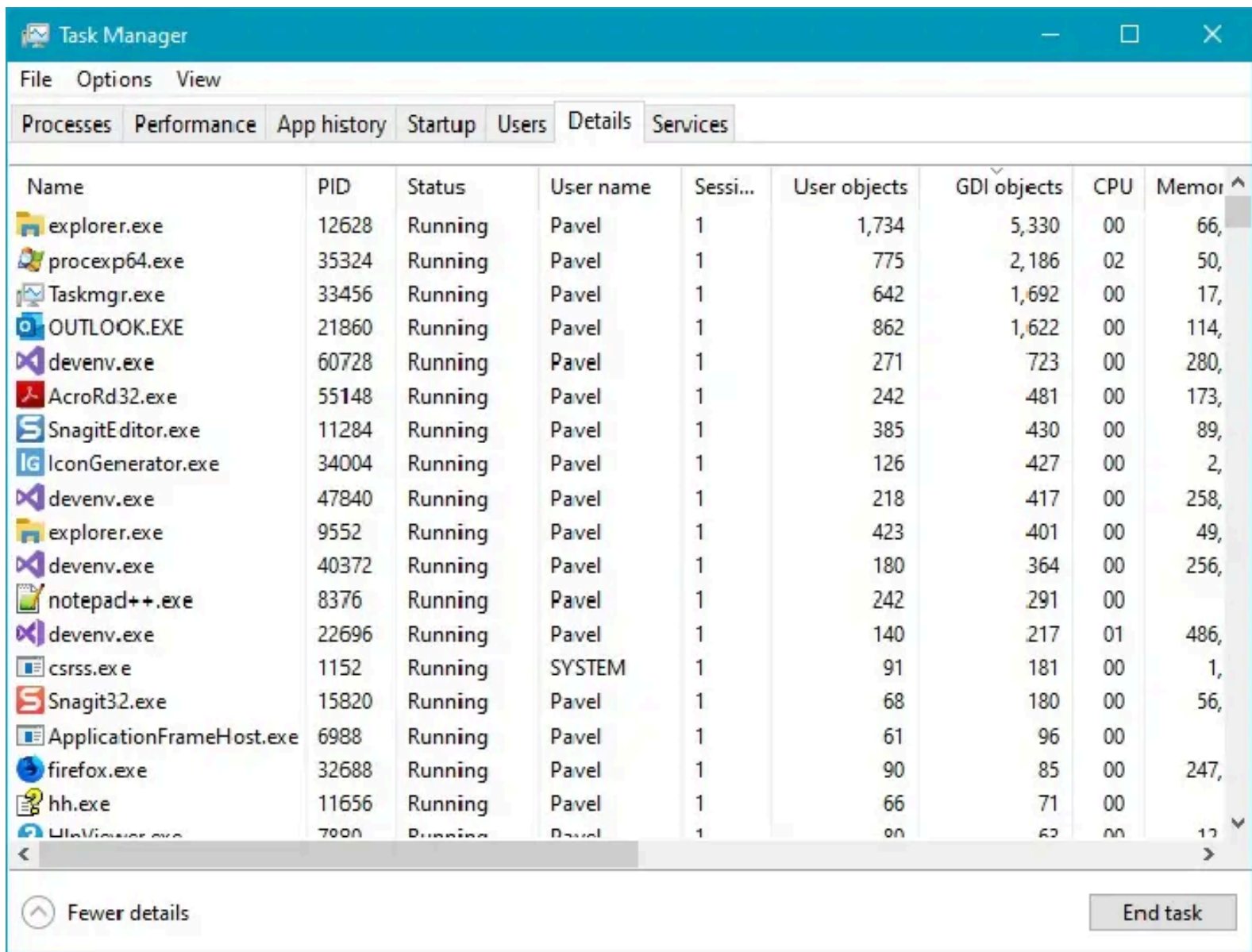
```
wil::unique_mapview_ptr<void> buffer(::MapViewOfFile(
    m_hSharedMem.get(), FILE_MAP_READ, 0, 0, 0));
if (!buffer) {
    AtlMessageBox(m_hWnd, L"Failed to map memory", IDR_MAINFRAME);
    return 0;
}

SetDlgItemText(IDC_TEXT, (PCWSTR)buffer.get());
```

Other Objects and Handles

Kernel objects are interesting in the context of system programming, and are the focus of this book. There are other common objects used in Windows, namely user objects and GDI objects. The following is a brief description of these objects and handles to such objects.

Task Manager can show the number of such objects for each process by adding the *User Objects* and *GDI Objects* columns, as shown in figure 2-21.



Name	PID	Status	User name	Sessi...	User objects	GDI objects	CPU	Memor
explorer.exe	12628	Running	Pavel	1	1,734	5,330	00	66,
procexp64.exe	35324	Running	Pavel	1	775	2,186	02	50,
Taskmgr.exe	33456	Running	Pavel	1	642	1,692	00	17,
OUTLOOK.EXE	21860	Running	Pavel	1	862	1,622	00	114,
devenv.exe	60728	Running	Pavel	1	271	723	00	280,
AcroRd32.exe	55148	Running	Pavel	1	242	481	00	173,
SnagitEditor.exe	11284	Running	Pavel	1	385	430	00	89,
IconGenerator.exe	34004	Running	Pavel	1	126	427	00	2,
devenv.exe	47840	Running	Pavel	1	218	417	00	258,
explorer.exe	9552	Running	Pavel	1	423	401	00	49,
devenv.exe	40372	Running	Pavel	1	180	364	00	256,
notepad++.exe	8376	Running	Pavel	1	242	291	00	
devenv.exe	22696	Running	Pavel	1	140	217	01	486,
csrss.exe	1152	Running	SYSTEM	1	91	181	00	1,
Snagit32.exe	15820	Running	Pavel	1	68	180	00	56,
ApplicationFrameHost.exe	6988	Running	Pavel	1	61	96	00	
firefox.exe	32688	Running	Pavel	1	90	85	00	247,
hh.exe	11656	Running	Pavel	1	66	71	00	
UltraViewer.exe	7000	Running	Pavel	1	80	62	00	12,

Figure 2-21: User and GDI object count

User Objects

User objects are Windows (HWND), Menus (HMENU) and hooks (HHOOK). Handles to these objects have the following attributes:

- No reference counting. The first caller that destroys a user object - it's gone.
- Handle values are scoped under a Window Station. A Window Station contains a clipboard, desktops and atom table. This means handles to these objects can be passed freely among all applications sharing a desktop, for instance.

The terms Window Station and desktop will be discussed later in this book. Atom tables will not, as these are related to the UI subsystem in Windows, which is not the focus of this book.

GDI Objects

The *Graphics Device Interface* (GDI) is the original graphics API in Windows and is still used today, even though there are richer and better APIs (Direct2D for example). Example GDI objects: device context (HDC), pen (HPEN), brush (HBRUSH), bitmap (HBITMAP) and others. Here are their attributes:

- No reference counting.
- Handles are valid only in the process in which they are created.
- Cannot be shared between processes.

Summary

In this chapter, we looked at kernel objects and the ways they can be accessed and shared by using handles. We did not look at any specific object type too closely, as these will be discussed in other chapters in more detail. In the next chapter, we'll delve into the most well-known of all kernel objects - the process.